

RECONSTRUCTION TOMOGRAPHIQUE

TIPE | Louis H. | 00000

Introduction



Inauguration of CT Scan Services in Fuvahmulah, Maldives, 2025, The President's Office

ÉTUDE THÉORIQUE DE LA TOMODENSITOMÉTRIE AXIALE

Comment concevoir un programme informatique capable de réaliser une tomodensitométrie axiale, et comment optimiser la reconstruction tomographique dans le but de diminuer l'exposition aux rayons X d'un sujet ?

Sommaire

- **Cadre de l’étude**
- **Méthodes classiques de tomographie**
- **Réseaux de neurones**
- **Amélioration de la tomographie**

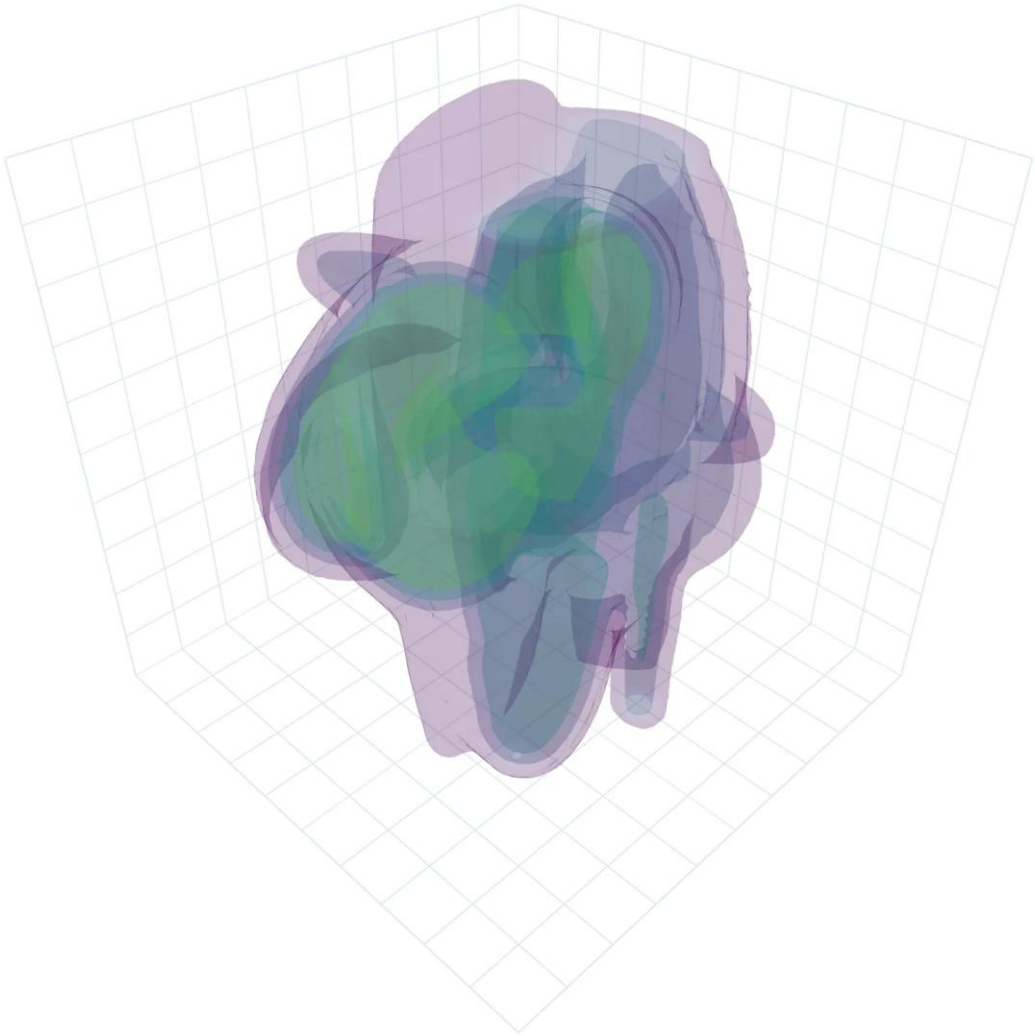
Cadre de l'étude

Idée générale | Transformée de RADON

Tomodensitométrie axiale

Idée générale

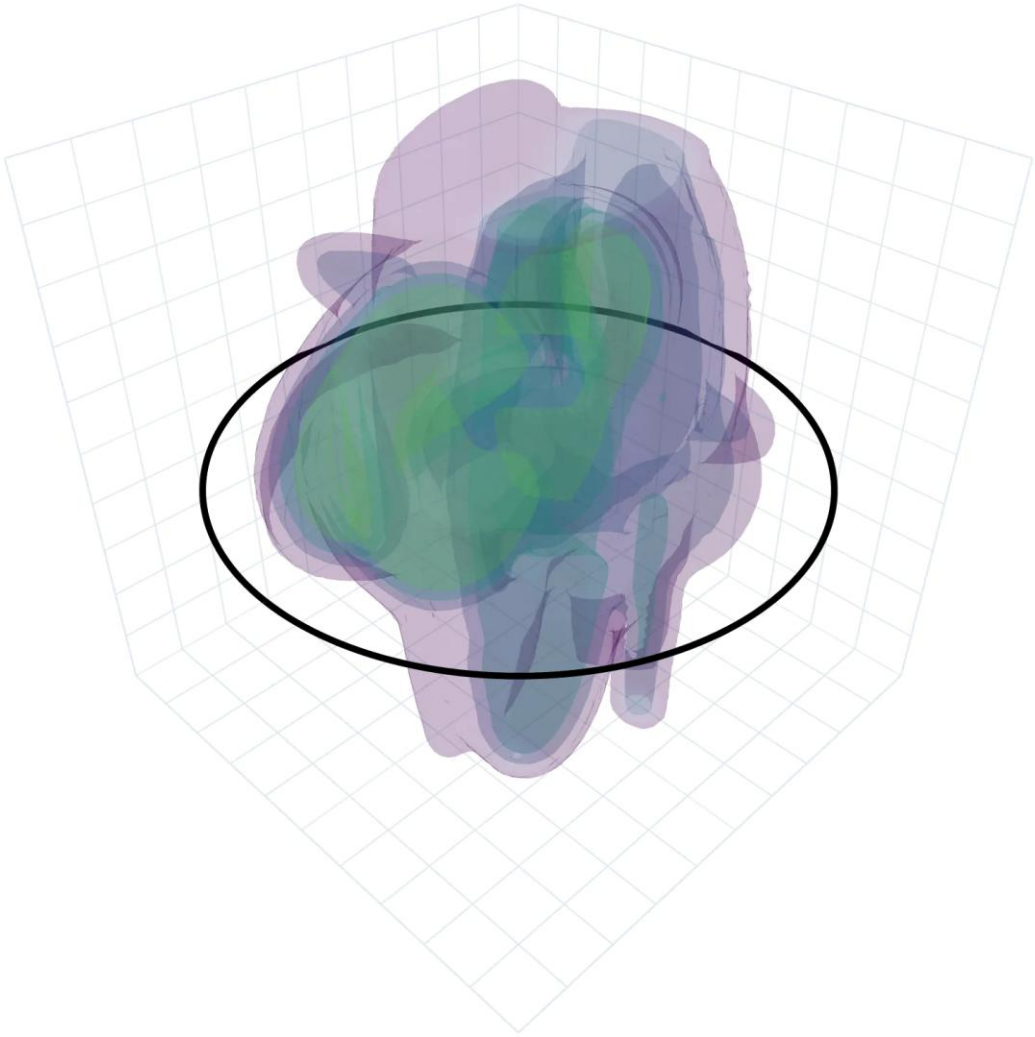
RECONSTRUCTION TOMOGRAPHIQUE



Tomodensitométrie axiale

Idée générale

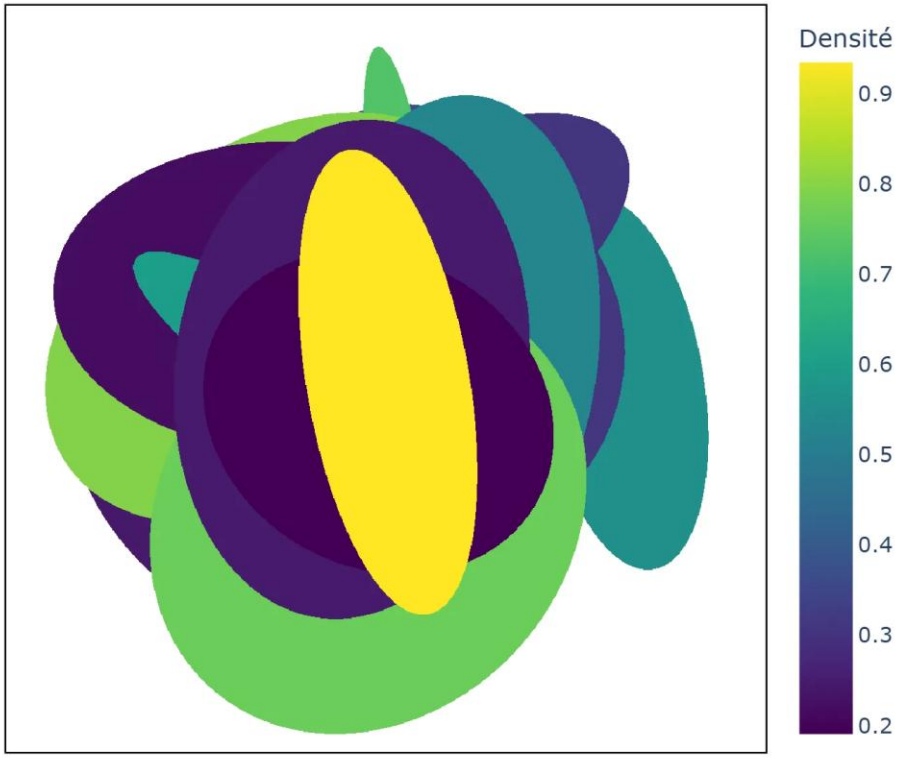
RECONSTRUCTION TOMOGRAPHIQUE



Tomodensitométrie axiale

Cadre de l'étude

Simulation d'un densité osseuse



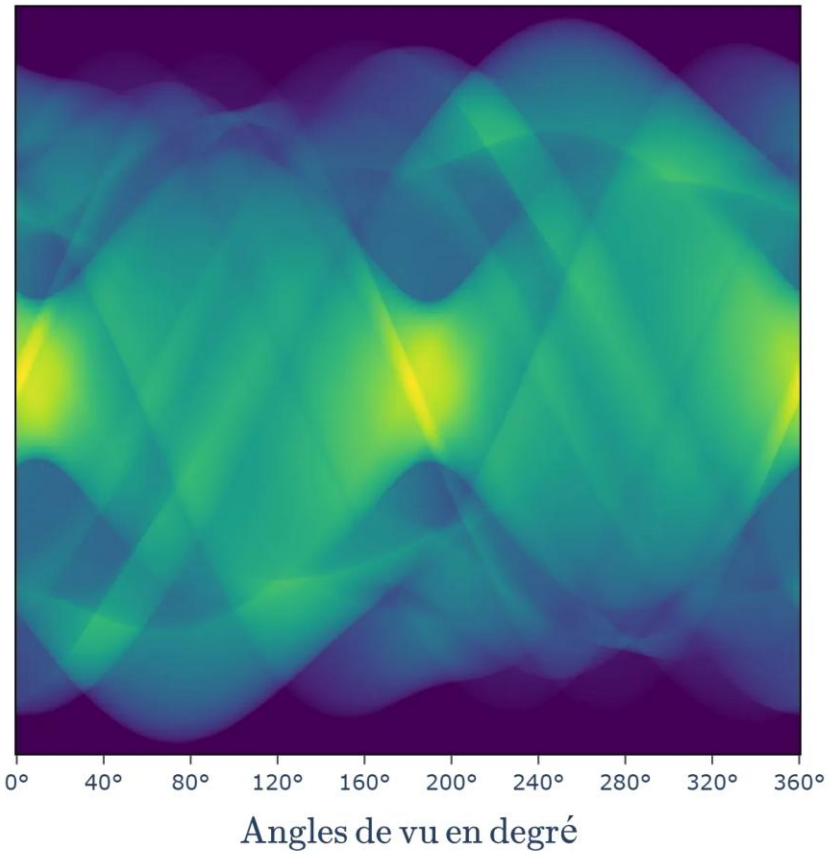
Le sujet

- 2 dimensions
- Superposition d'ellipsoïdes aléatoires
- Densité aléatoire $\sim \mathbb{U}\left(\left[\frac{1}{10}\,;1\right]\right)$
- Représente un « organe »

Tomodensitométrie axiale

Transformée de Radon

Sinogramme de la densité osseuse



Transformée de Radon

Pour chaque $\theta \in \Theta, r \in [0, l]$

$$\mathrm{R}f(r,\theta) = \int_{-\infty}^{+\infty} f\begin{pmatrix} r\cos\theta + z\sin\theta \\ r\sin\theta - z\cos\theta \end{pmatrix} \mathrm{d}z$$

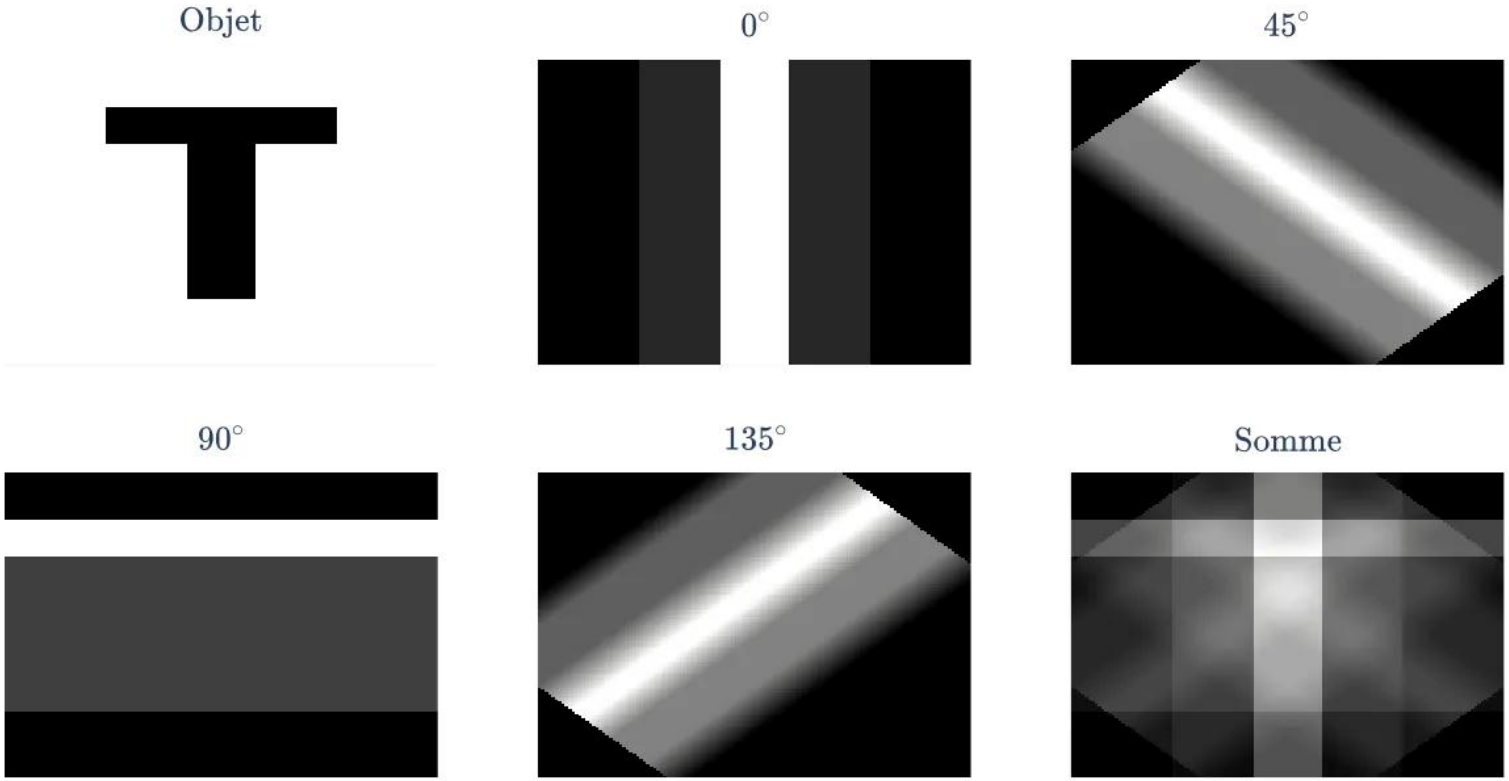
Cadre de l'étude

- $r \in [0, 100[$
- θ de l'ordre de 10 à 1000 prises de vues
- 5 à 30 ellipsoïdes

Méthodes classiques

Rétroprojection | Reconstruction itérative

Rétroprojection

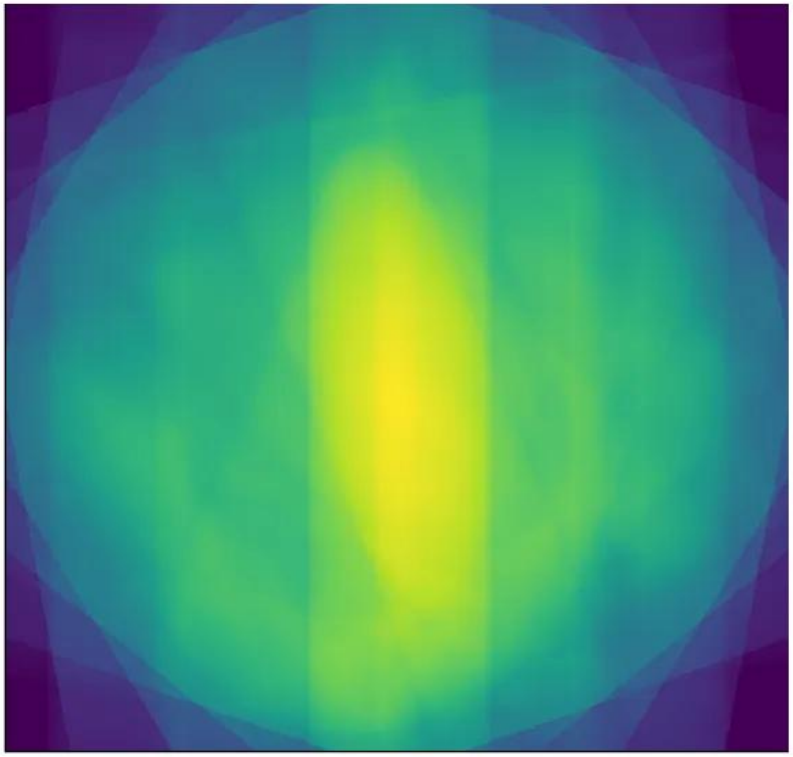


Rétroprojection

Influence du nombre de prises de vues

Reconstruction de la densité osseuse

Rétroprojection - 10 prises de vues

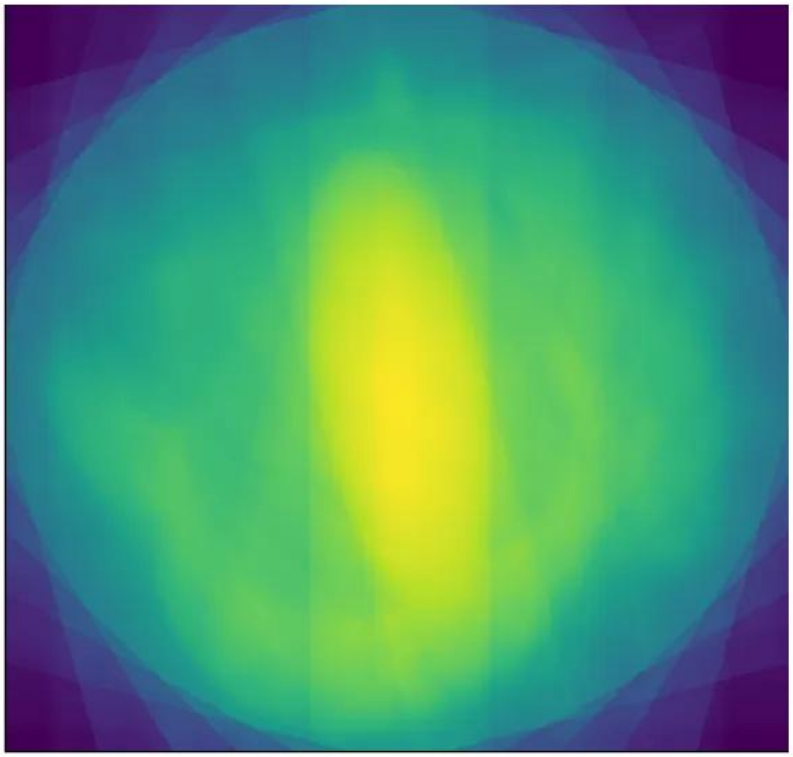


Rétroprojection

Influence du nombre de prises de vues

Reconstruction de la densité osseuse

Rétroprojection - 25 prises de vues

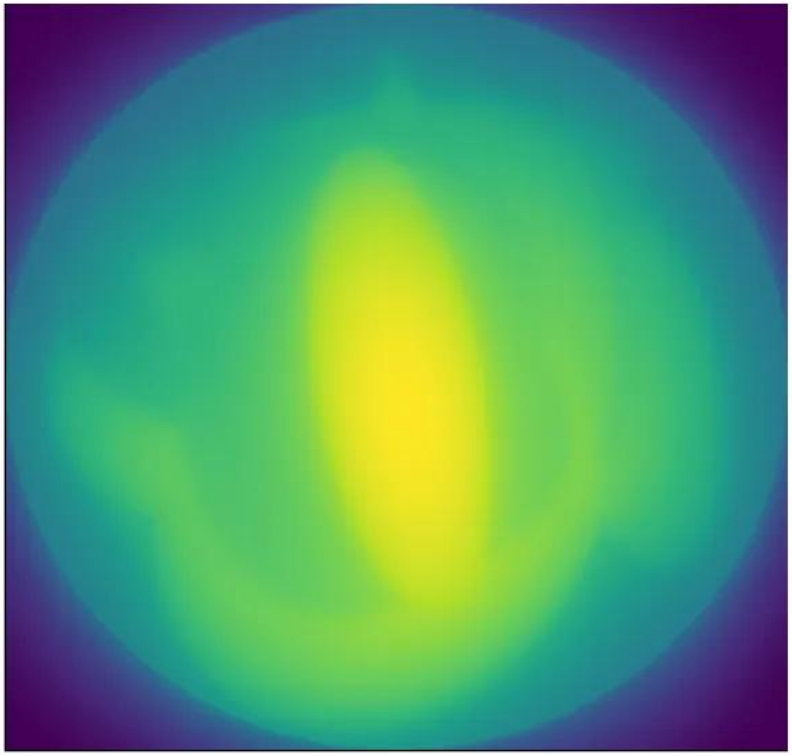


Rétroprojection

Influence du nombre de prises de vues

Reconstruction de la densité osseuse

Rétroprojection - 100 prises de vues

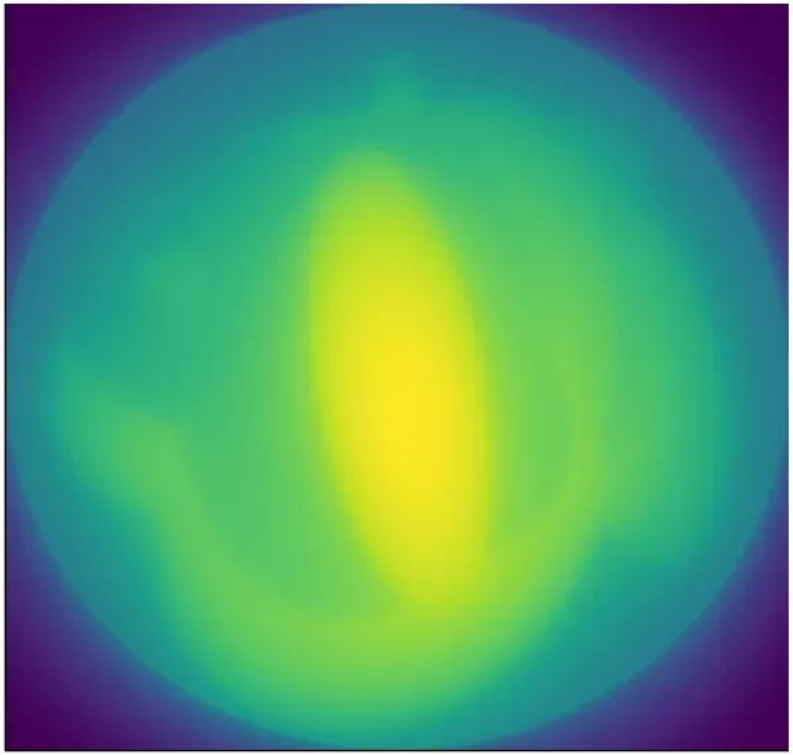


Rétroprojection

Influence du nombre de prises de vues

Reconstruction de la densité osseuse

Rétroprojection - 360 prises de vues

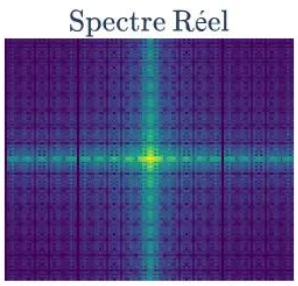


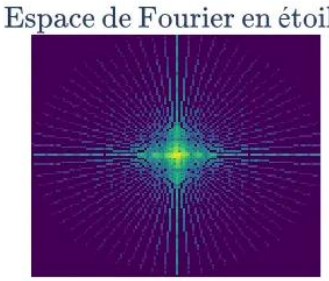
Amélioration

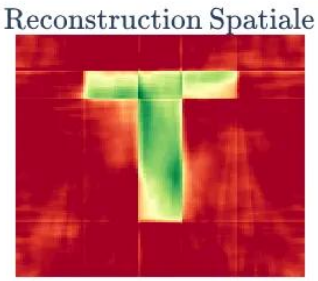
Reconstruction itérative et reconstruction en domaine de FOURIER

Reconstruction en domaine de FOURRIER









Reconstruction itérative

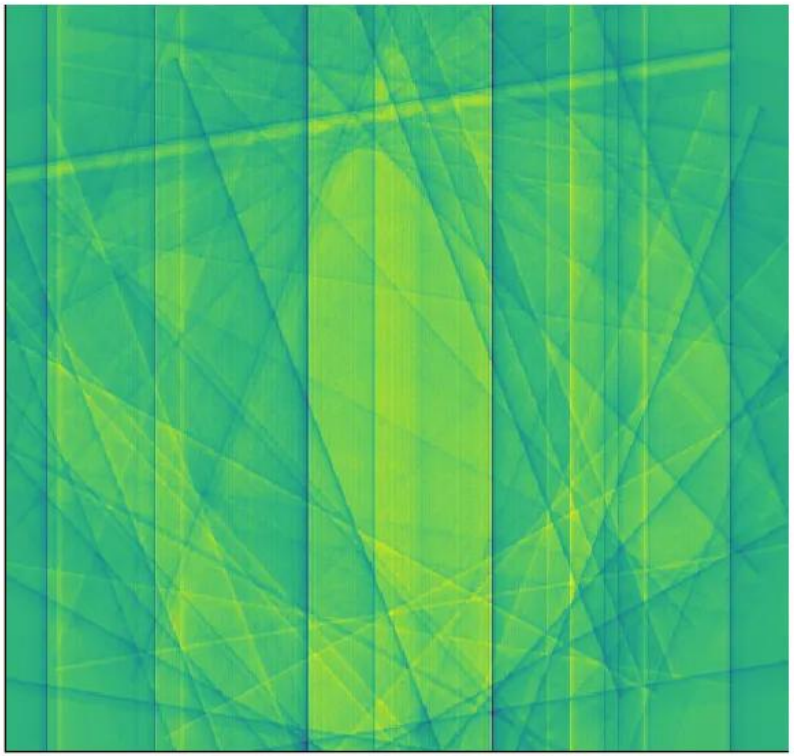
- Problème d'optimisation numérique
- $X_{k+1} = X_k + \alpha A^T (B - AX_k)$
- AX_k : transformée de RADON de X_k

Reconstruction en domaine de FOURIER

Influence du nombre de prises de vues

Reconstruction de la densité osseuse

Reconstruction dans le domaine de Fourier - 10 prises de vues

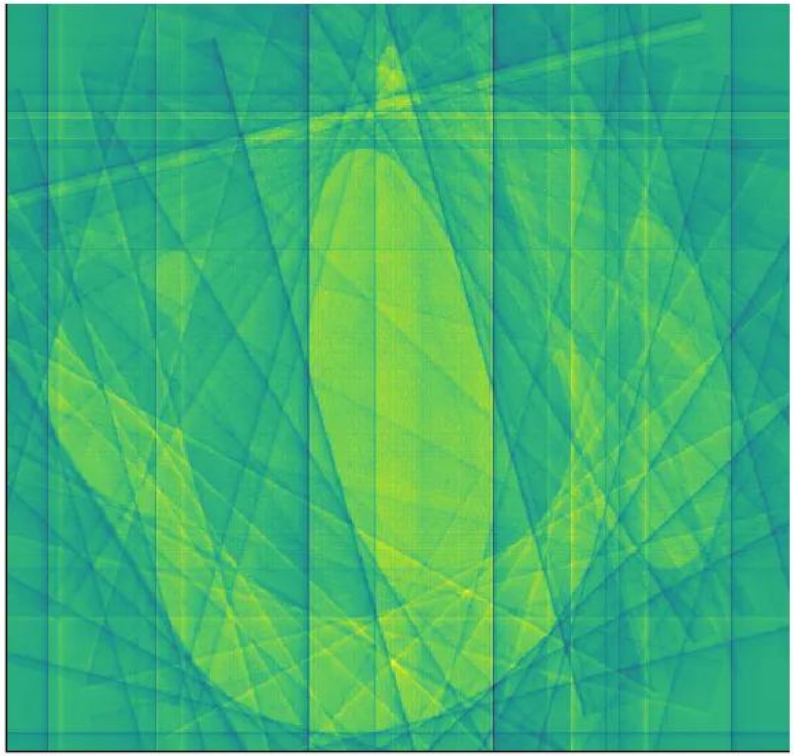


Reconstruction en domaine de FOURIER

Influence du nombre de prises de vues

Reconstruction de la densité osseuse

Reconstruction dans le domaine de Fourier - 25 prises de vues

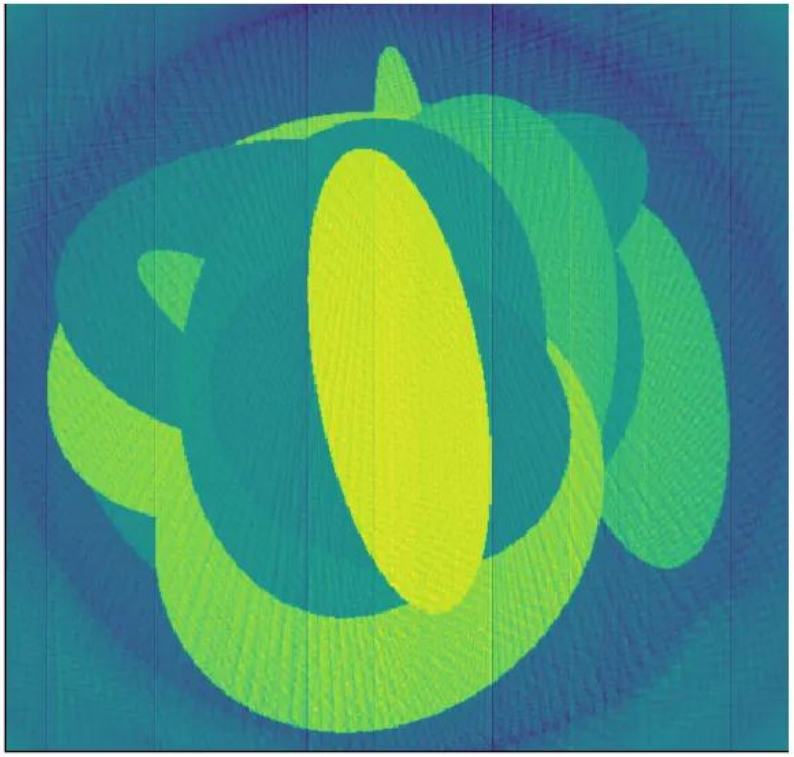


Reconstruction en domaine de FOURIER

Influence du nombre de prises de vues

Reconstruction de la densité osseuse

Reconstruction dans le domaine de Fourier - 100 prises de vues

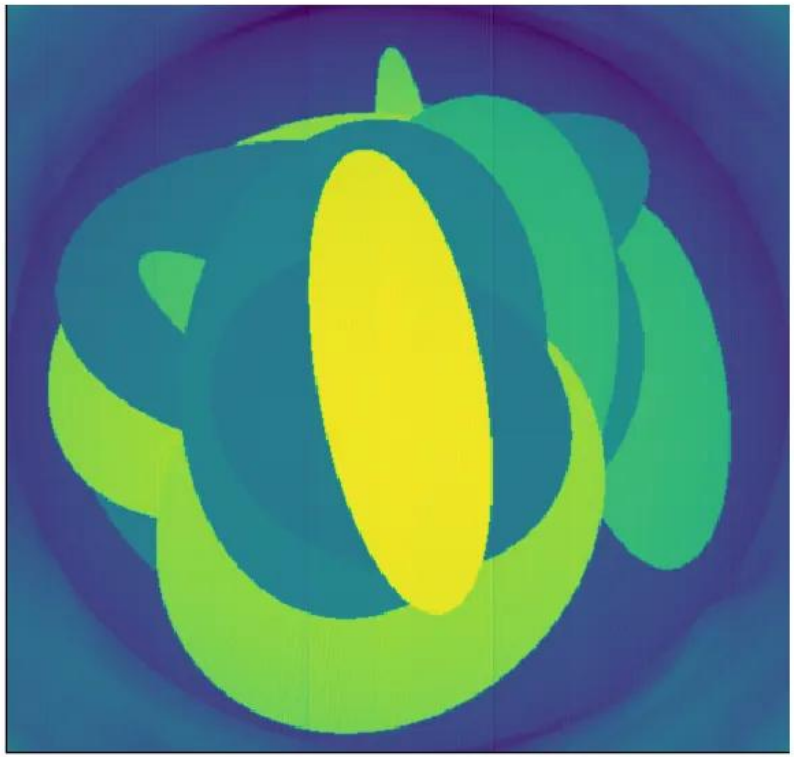


Reconstruction en domaine de FOURIER

Influence du nombre de prises de vues

Reconstruction de la densité osseuse

Reconstruction dans le domaine de Fourier - 360 prises de vues

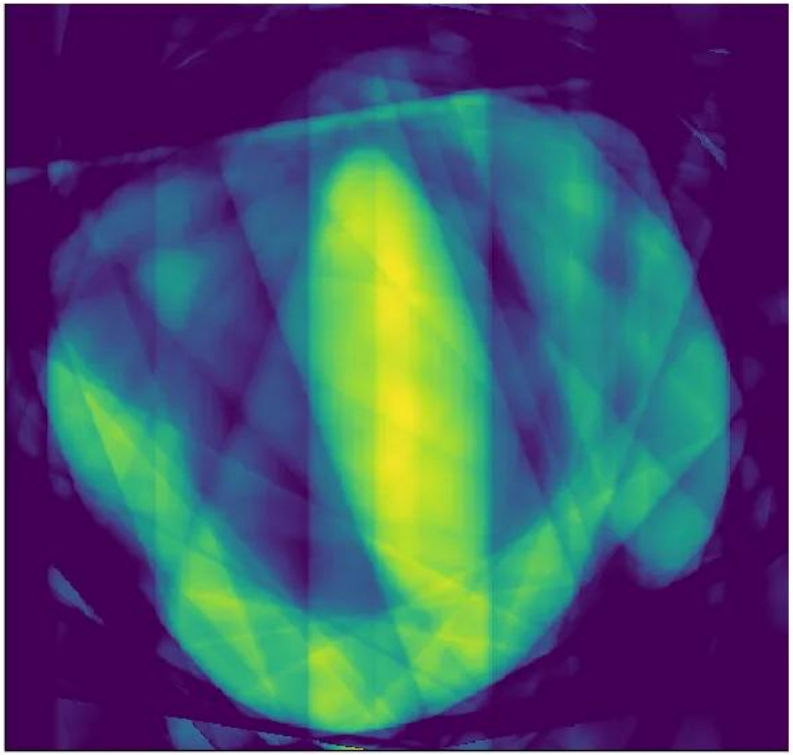


Reconstruction itérative

Influence du nombre de prises de vues

Reconstruction de la densité osseuse

Reconstruction itérative (5 itérations) - 10 prises de vues

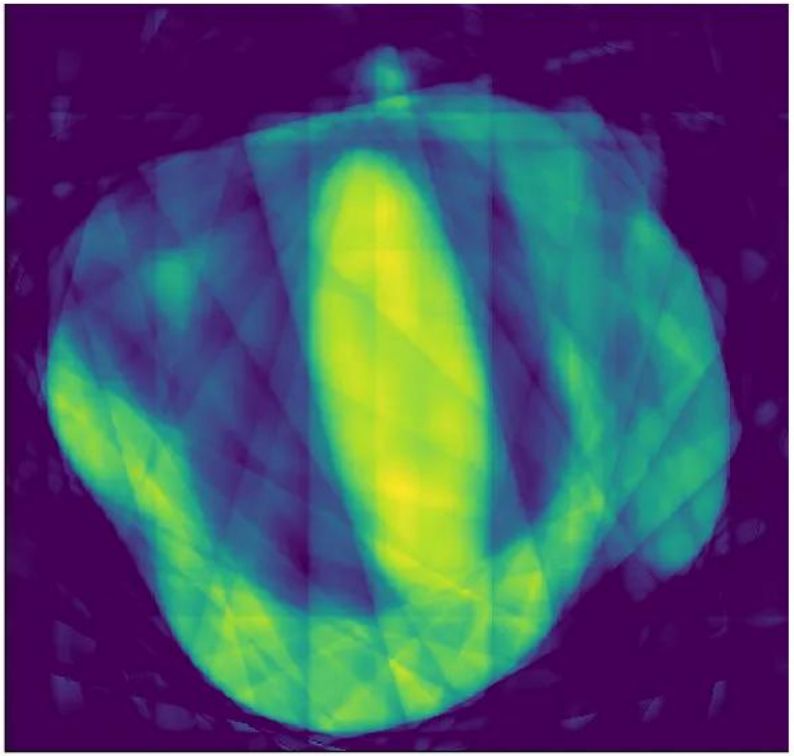


Reconstruction itérative

Influence du nombre de prises de vues

Reconstruction de la densité osseuse

Reconstruction itérative (5 itérations) - 25 prises de vues

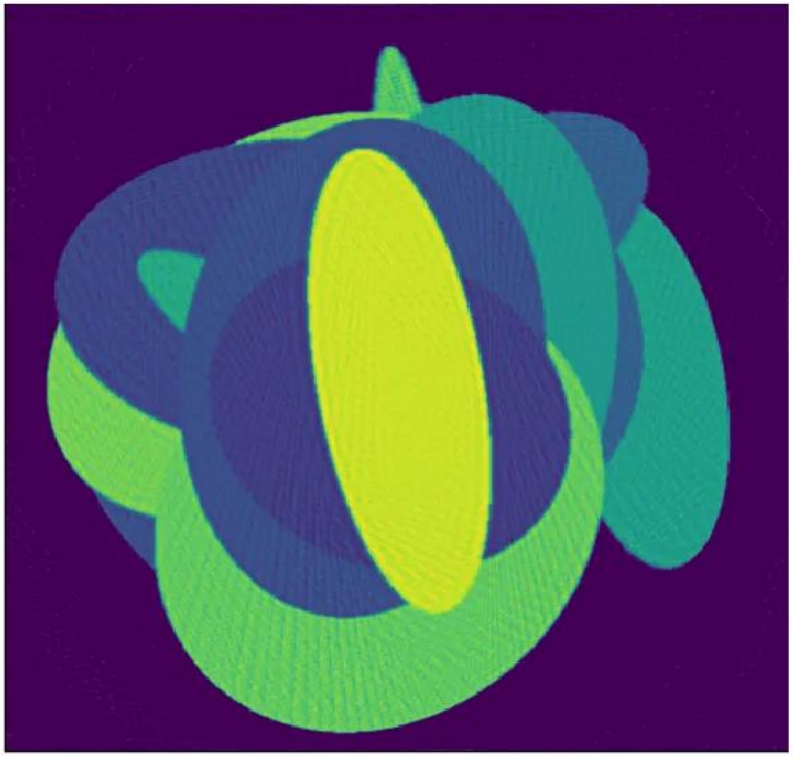


Reconstruction itérative

Influence du nombre de prises de vues

Reconstruction de la densité osseuse

Reconstruction itérative (5 itérations) - 100 prises de vues



Reconstruction itérative

Influence du nombre de prises de vues

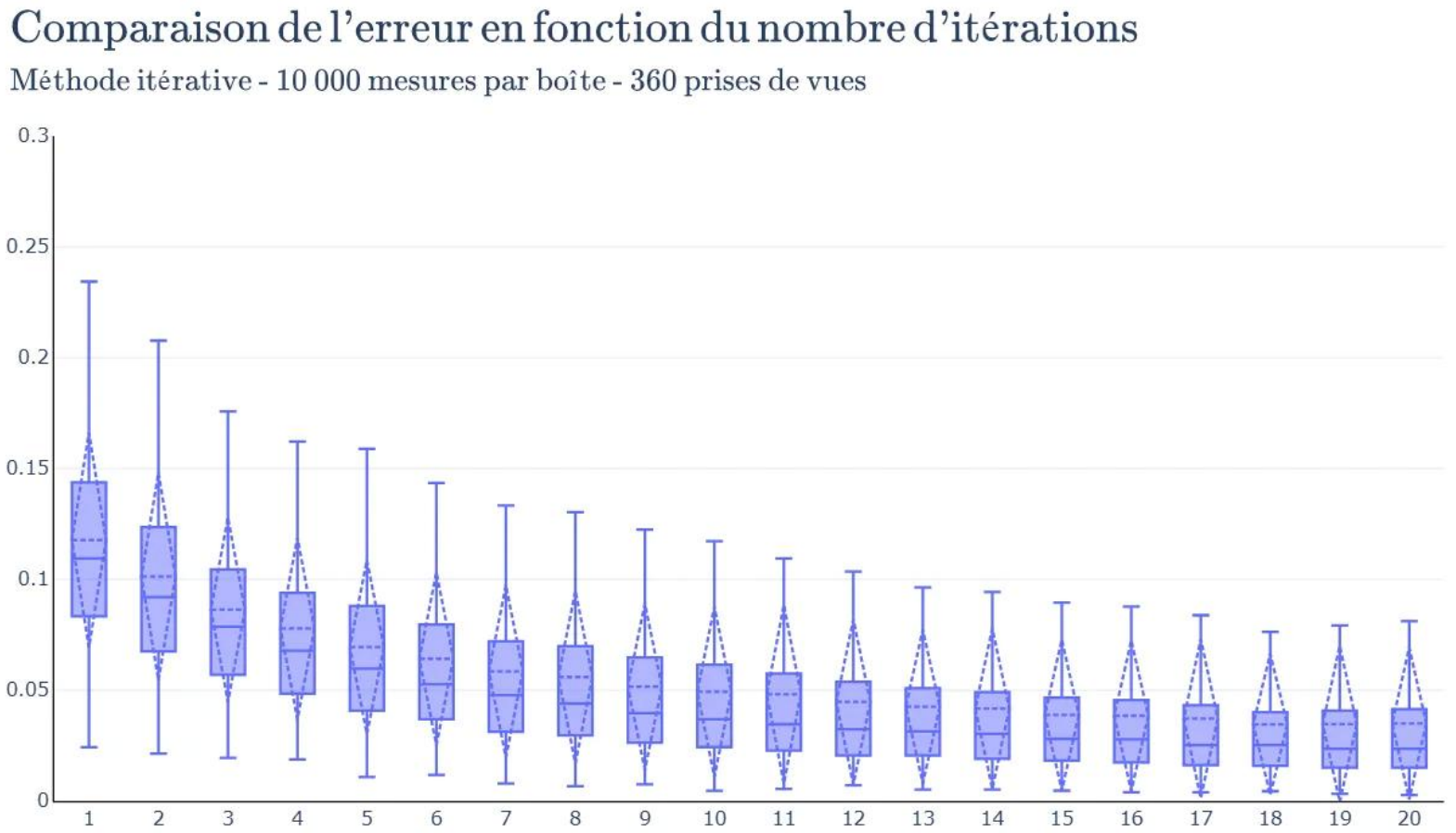
Reconstruction de la densité osseuse

Reconstruction itérative (5 itérations) - 360 prises de vues



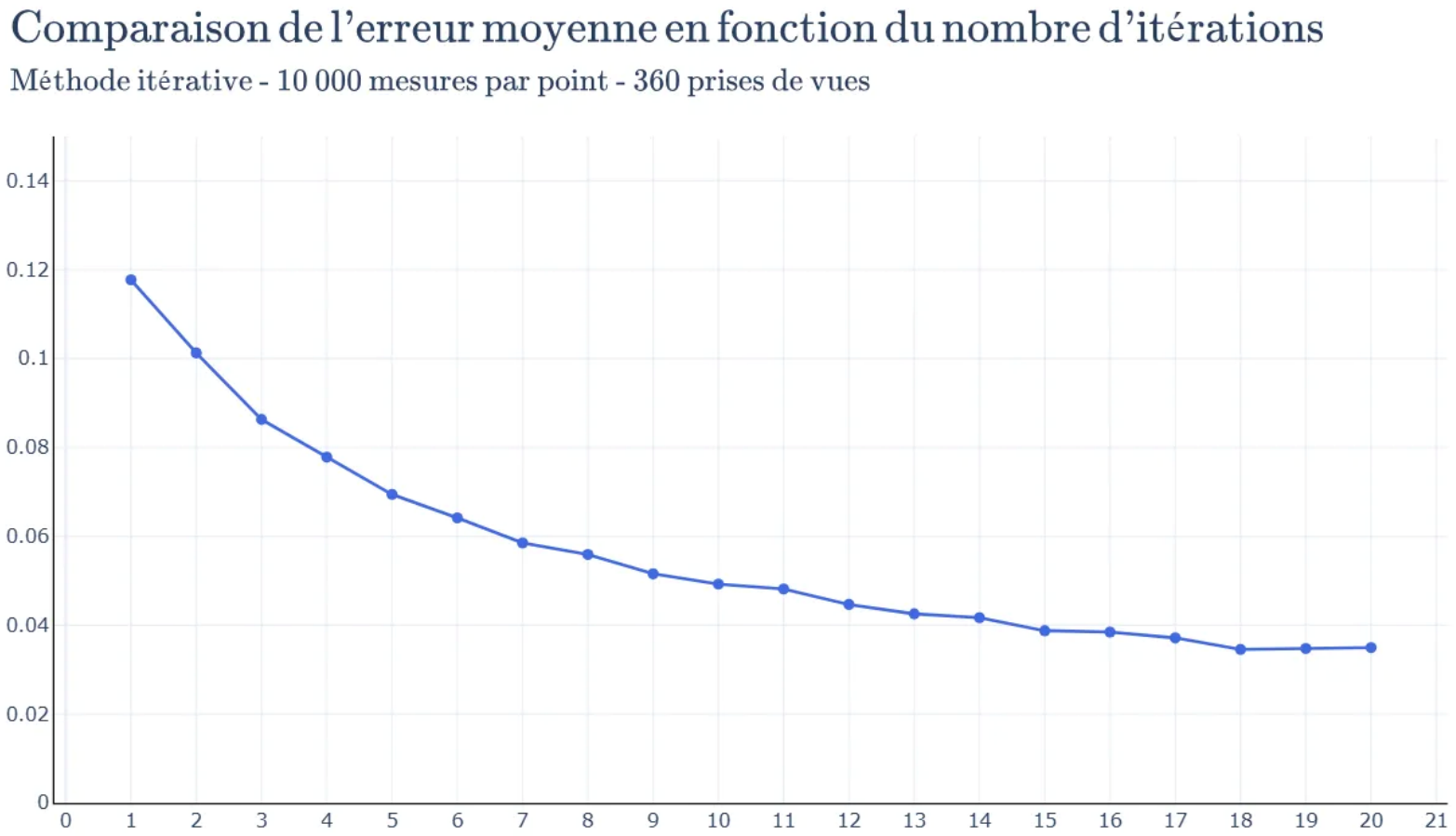
Reconstruction itérative

Nombre d'itérations

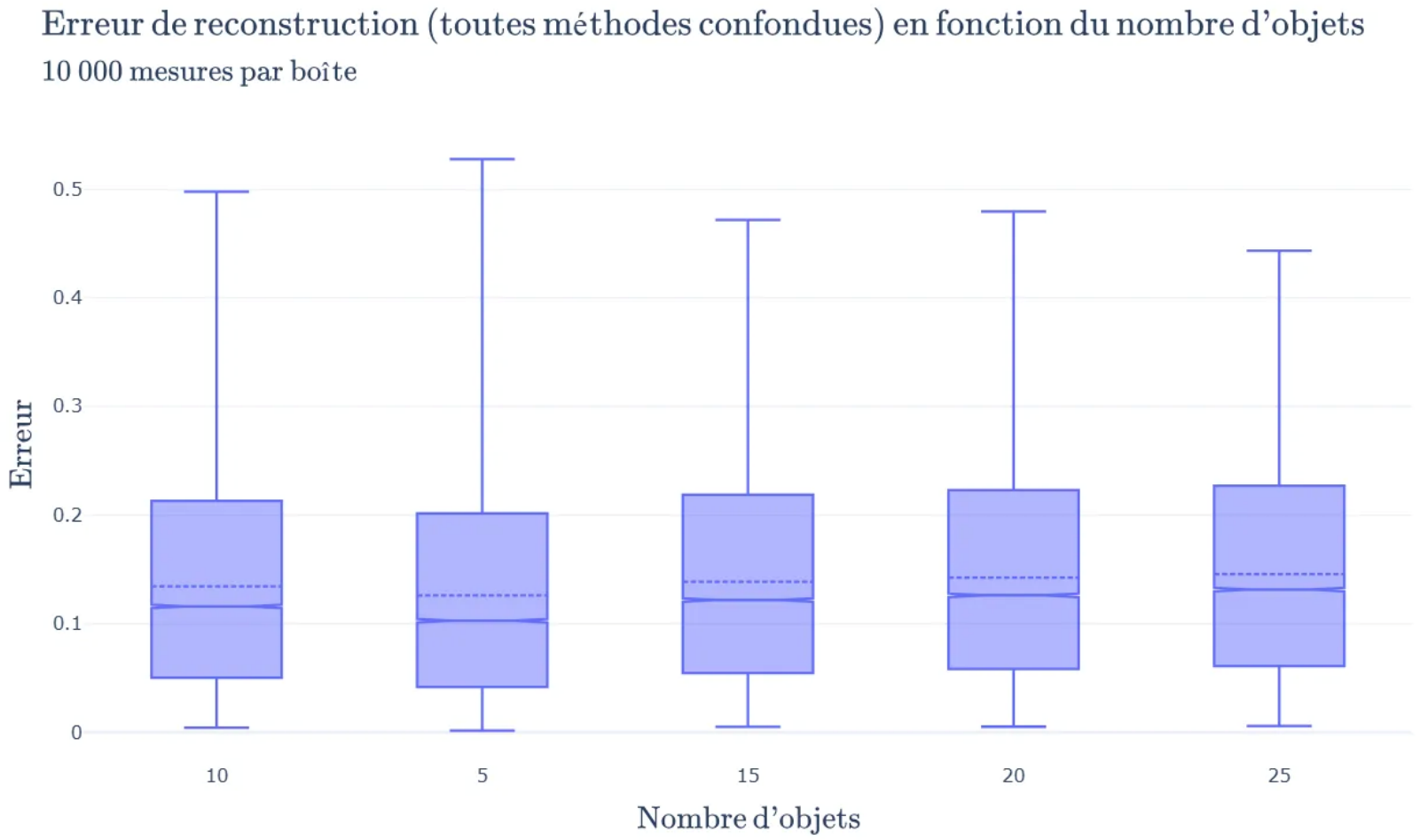


Reconstruction itérative

Nombre d'itérations

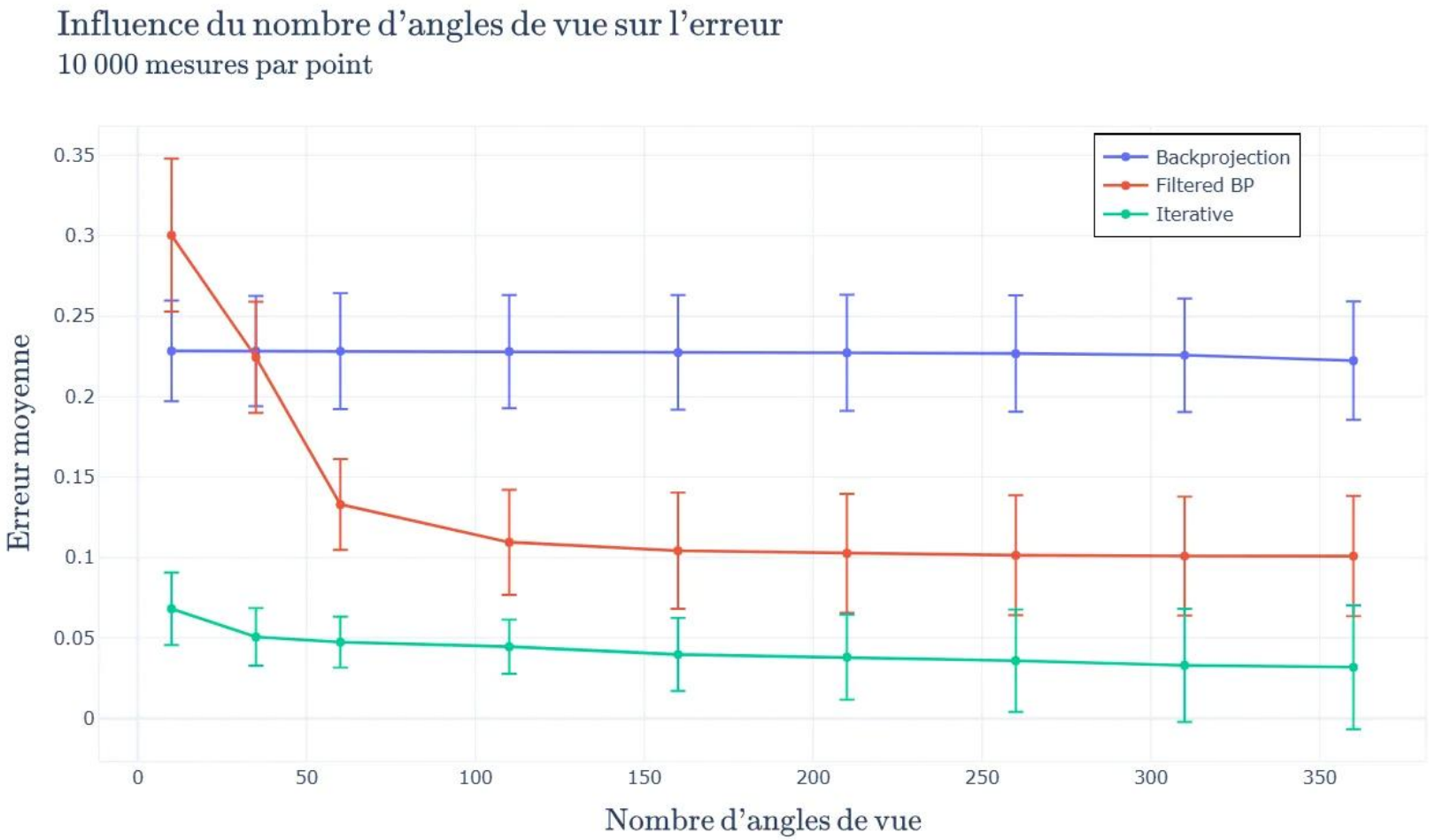


Importance du nombre d'ellipsoïde

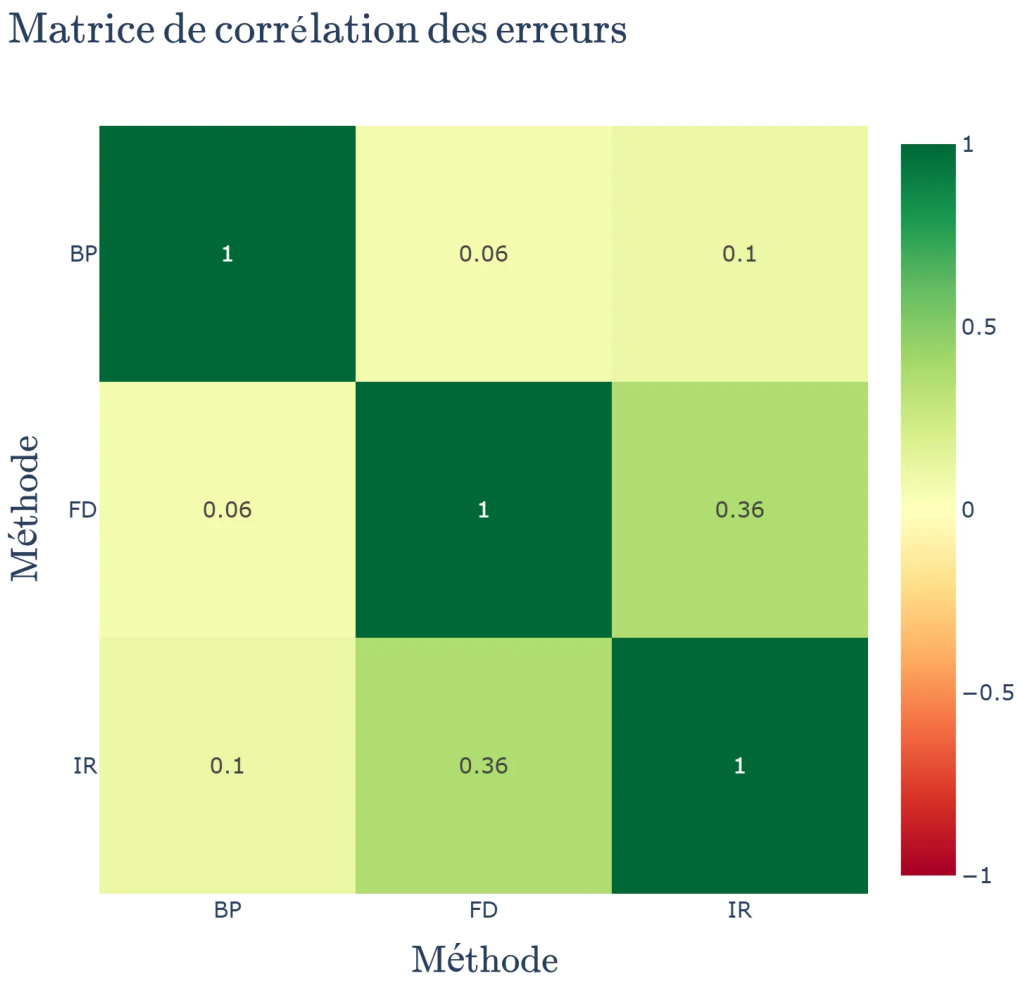


Comparaison des trois méthodes

Nombre d'itérations



Matrice de corrélation des erreurs



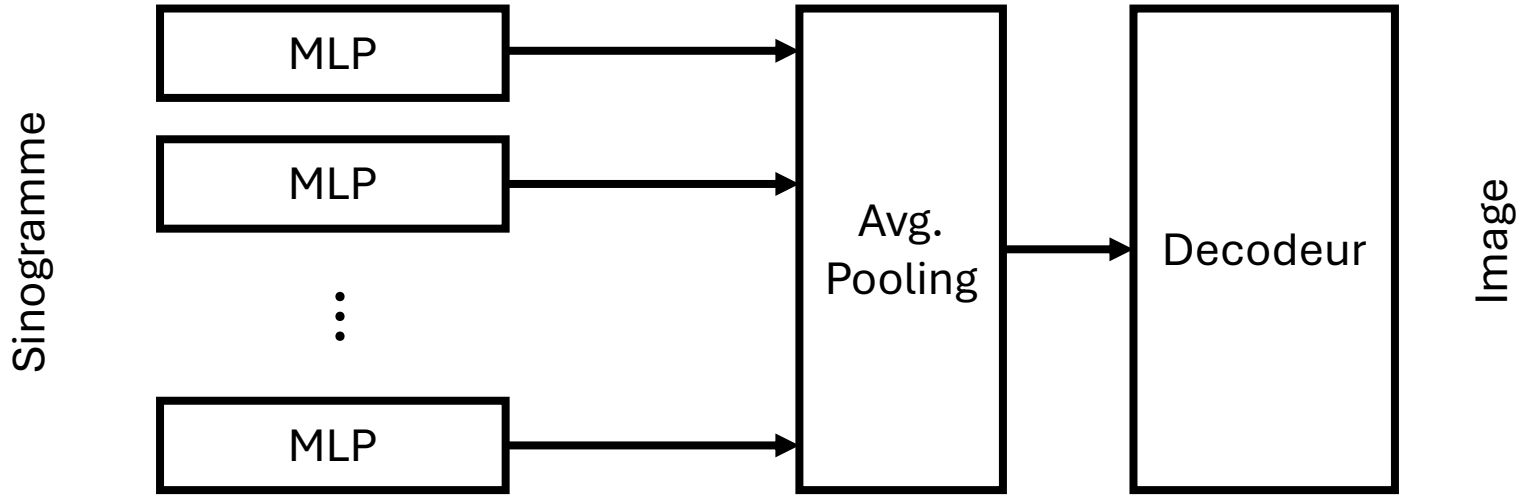
Réseaux de neurones

Rétroprojection | Reconstruction itérative

Première architecture

Inspiré de l'architecture PointNet[1]

- Perceptron multicouche pour chaque projection
- Agrégation
- Décodeur



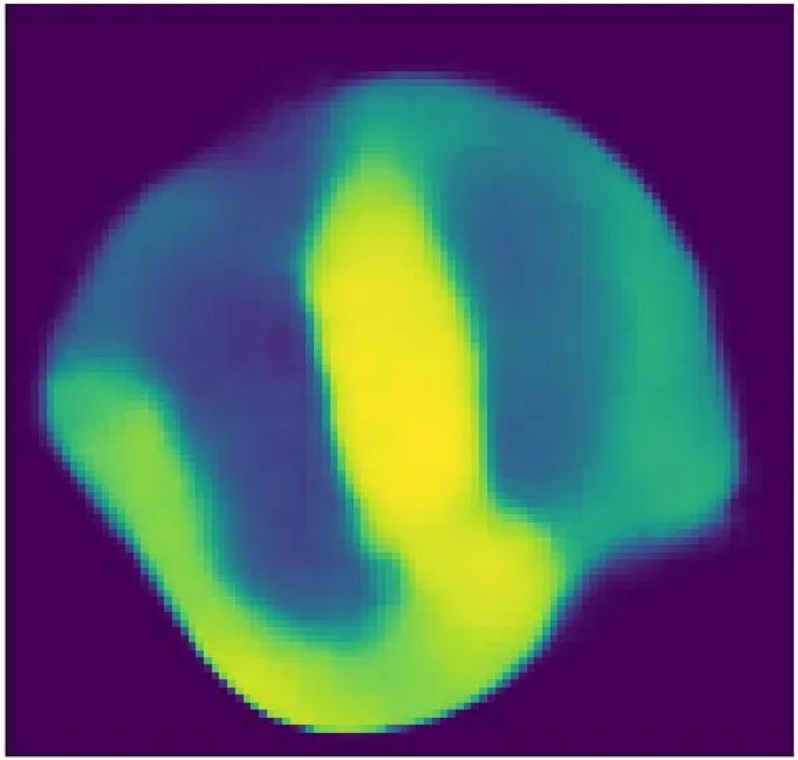
[1] Charles R. Qi, Hao Su, Kaichun Mo, & Leonidas J. Guibas. (2017). PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation.

Première architecture

Résultat

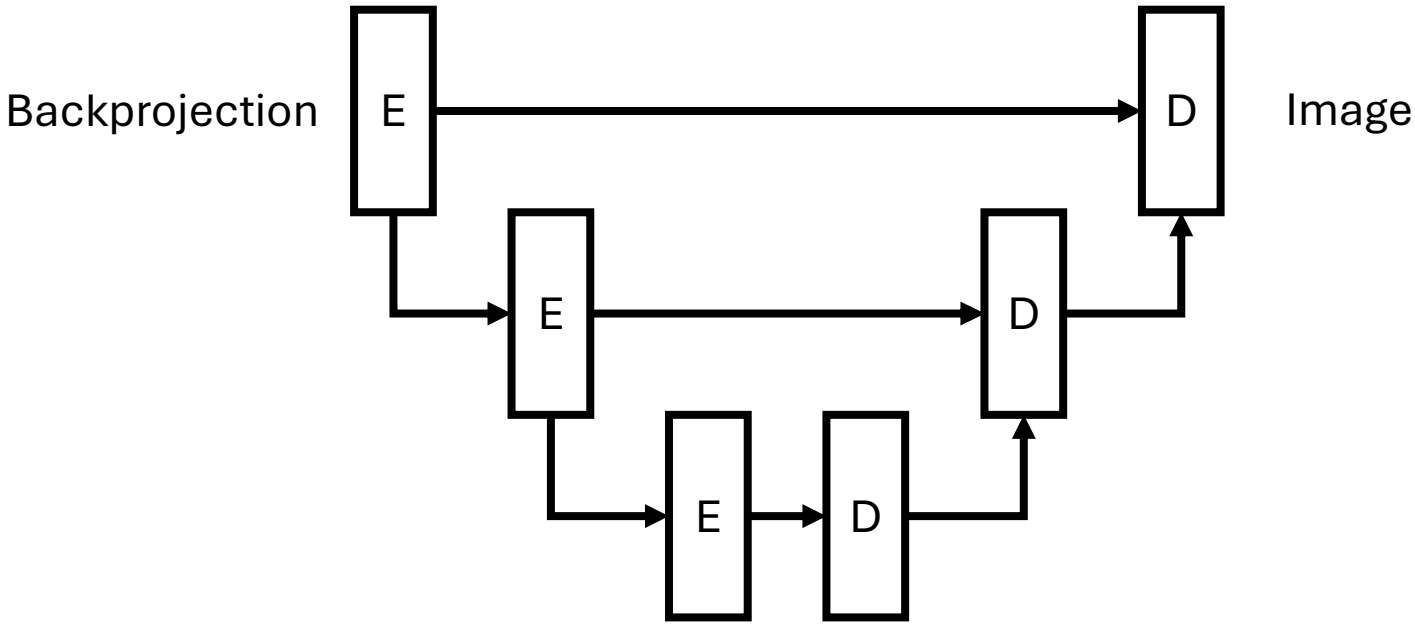
Reconstruction de la densité osseuse

Réseau de neurone - 360 prises de vues



Seconde architecture

Architecture FBPConvNet[2]



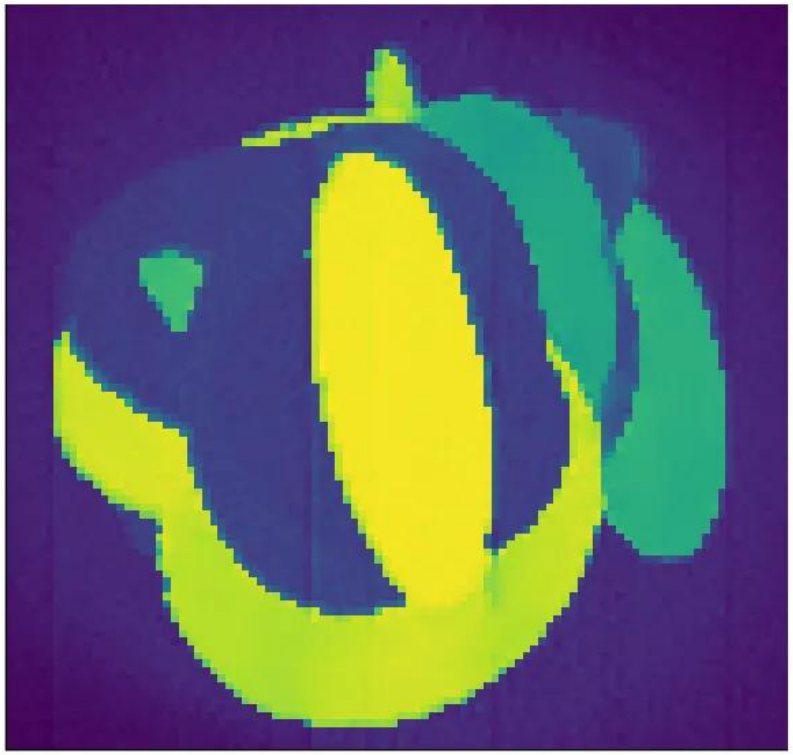
[2] Wang J, Liang J, Cheng J, Guo Y, Zeng L. Deep learning based image reconstruction algorithm for limited-angle translational computed tomography. PLoS One. 2020 Jan 6

Seconde architecture

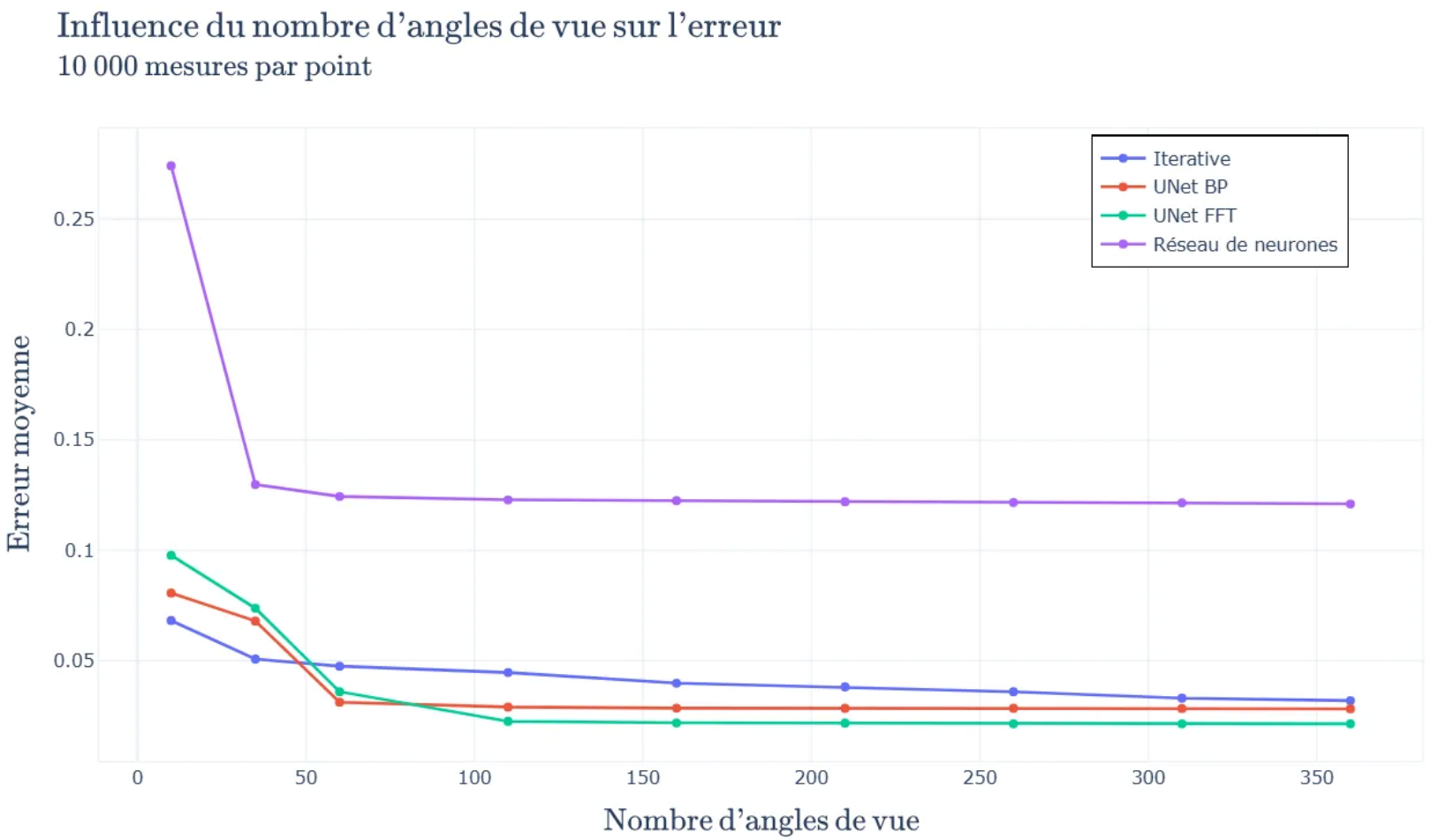
Résultat

Reconstruction de la densité osseuse

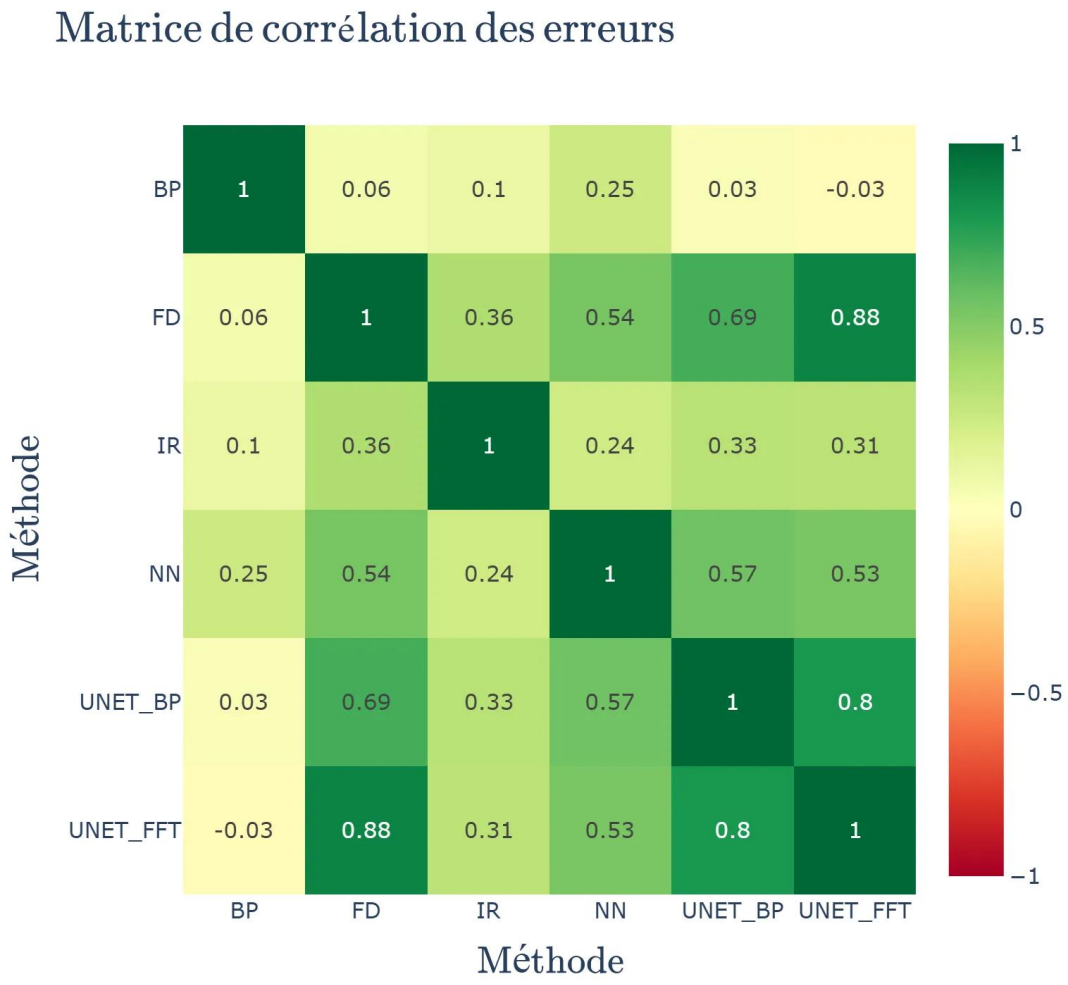
Réseau de neurone - 100 prises de vues



Comparaison des méthodes

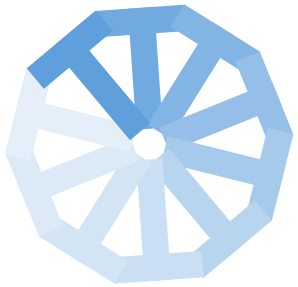


Matrice de corrélation des erreurs



ÉTUDE THÉORIQUE DE LA TOMODENSITOMÉTRIE AXIALE.

Comment concevoir un programme informatique capable de réaliser une tomodensitométrie axiale, et comment optimiser la reconstruction tomographique dans le but de diminuer l'exposition aux rayons X d'un sujet ?



RECONSTRUCTION TOMOGRAPHIQUE

TIPE | Louis H. | 00000

Annexe

Code source | Python

Code source	Diapositive
- tomography	
- `__init__.py`	40
- `Monte_carlo_analysis.py`	46
- `Tomograhpic_reconstruction.py`	53
- `utils.py`	58
- results	
- `1 illustrate problem.ipynb`	60
- `2 iterative_reconstruction_parameter.py`	67
- `2 mathematical methods.ipynb`	70
- `3 methods_comparison.py`	82
- `3 methods_comparison.ipynb`	94
- `4 illustration.py`	102
- neural_networks	
- `__init__.py`	108
- `direct_reconstruction.py`	111
- `unet.py`	118

tomography/__init__.py

```
"""
All functions related to tomography and tomographic reconstruction
"""
import random
from typing import Callable

import numpy as np
from scipy.ndimage import rotate

from tomography.utils import limit_circle

"""
2 dimensional random scan
"""


def radon_transform(image: np.ndarray, angles: np.ndarray) -> np.ndarray:
    sinogram = []

    for angle in angles:
        rotated = rotate(image, angle, reshape=False, order=1)
        projection = np.sum(rotated, axis=0)
        sinogram.append(projection)
    return np.array(sinogram).T   # shape: (N, n_angles)


def random_continuous_image(height: int,
                            width: int,

                            amount: int,

                            min_value: float = 0.0,
                            max_value: float = 1.0,

                            ellipse_param: Callable[[], float] = lambda: random.gauss(0.5, 0.1),
                            ellipse_position: Callable[[], float] = lambda: random.gauss(0, 0.25),
                            *,
                            alpha: float = 5.0,

                            ) -> np.ndarray:
    # Grille normalisée sur [-1, 1]
```

```
    x = np.linspace(-1, 1, width)
    y = np.linspace(-1, 1, height)
    mg_x, mg_y = np.meshgrid(x, y)

    # Masque circulaire (optionnel)
    lc = limit_circle(height, width)

    image = np.zeros((height, width), dtype=float)

    def generate_gaussian():
        # Paramètres ellipse = variances en x et y
        a, b = ellipse_param(), ellipse_param()
        x0, y0 = ellipse_position(), ellipse_position()
        angle = random.uniform(0, 2 * np.pi)

        # Rotation du repère
        new_x = np.cos(angle) * (mg_x - x0) + np.sin(angle) * (mg_y - y0)
        new_y = -np.sin(angle) * (mg_x - x0) + np.cos(angle) * (mg_y - y0)

        new_x = new_x * alpha
        new_y = new_y * alpha

        # Fonction gaussienne 2D avec axes a, b
        gaussian = np.exp(- (new_x ** 2 / (2 * a ** 2) + new_y ** 2 / (2 * b ** 2)))
        return gaussian

    for _ in range(amount):
        gaussian = generate_gaussian()
        amplitude = np.random.uniform(min_value, max_value)
        image += amplitude * gaussian * lc   # on somme, au lieu d'écraser

    image = (image - np.min(image)) / np.max(image) * max_value + min_value

    return image


def random_discrete_image(height: int,
                          width: int,

                          amount: int,
```

```
                          min_value: float = 0.1,
                          max_value: float = 1.0,

                          ellipse_param: Callable[[], float] = lambda: random.gauss(0.5, 0.75),
                          ellipse_position: Callable[[], float] = lambda: random.gauss(0, 0.25)

                          ) -> np.ndarray:
    image = np.zeros((height, width))

    x = np.linspace(-1, 1, width)
    y = np.linspace(-1, 1, height)

    mg_x, mg_y = np.meshgrid(x, y)

    lc = limit_circle(height, width)

    def generate_ellipse():
        # generate ellipses that satisfy some properties
        a, b = ellipse_param(), ellipse_param()
        x0, y0 = ellipse_position(), ellipse_position()
        angle = random.uniform(- np.pi * 2, np.pi * 2)

        new_x = np.cos(angle) * (mg_x - x0) + np.sin(angle) * (mg_y - y0)
        new_y = - np.sin(angle) * (mg_x - x0) + np.cos(angle) * (mg_y - y0)

        return new_x ** 2 / a ** 2 + new_y ** 2 / b ** 2

    for _ in range(amount):
        ellipse = generate_ellipse()

        while np.any(ellipse[~lc] < 1):
            ellipse = generate_ellipse()

        value = np.random.uniform(min_value, max_value)

        image[(ellipse <= 1) & lc] = value

    return image


"""
```

```
3 dimensional random scan
"""


def random_discrete_image_3d(height: int,
                             width: int,
                             depth: int,

                             amount: int,

                             min_value: float = 0.1,
                             max_value: float = 1.0,

                             ellipse_param: Callable[[], float] = lambda: random.gauss(0.5, 0.75),
                             ellipse_position: Callable[[], float] = lambda: random.gauss(0, 0.25)

                             ) -> np.ndarray:
    volume = np.zeros((height, width, depth))

    x = np.linspace(-1, 1, width)
    y = np.linspace(-1, 1, height)
    z = np.linspace(-1, 1, depth)

    mg_x, mg_y, mg_z = np.meshgrid(x, y, z)

    lc = (mg_x ** 2 + mg_y ** 2 + (mg_z / 5) ** 2) <= 1

    def generate_ellipse():
        # generate ellipses that satisfy some properties
        a, b, c = ellipse_param(), ellipse_param(), ellipse_param()
        x0, y0, z0 = ellipse_position(), ellipse_position(), ellipse_position()
        angle = random.uniform(- np.pi * 2, np.pi * 2)

        new_x = np.cos(angle) * (mg_x - x0) + np.sin(angle) * (mg_y - y0)
        new_y = - np.sin(angle) * (mg_x - x0) + np.cos(angle) * (mg_y - y0)

        return (new_x / a) ** 2 + (new_y / b) ** 2 + ((mg_z - z0) / c) ** 2

    for _ in range(amount):
        ellipse = generate_ellipse()
```

```
        while np.any(ellipse[~lc] < 1):
            ellipse = generate_ellipse()

        value = np.random.uniform(min_value, max_value)

        volume[(ellipse <= 1)] = value

    return volume


if __name__ == '__main__':
    """import matplotlib.pyplot as plt
    import matplotlib

    matplotlib.use("TkAgg")

    V = random_discrete_image_3d(25, 25, 25, 20)

    for i in range(25):
        plt.matshow(V[i])

        plt.show()"""

    import matplotlib.pyplot as plt
    import matplotlib

    matplotlib.use("TkAgg")

    plt.matshow(random_discrete_image(100, 100, 10))

    plt.show()
```

tomography/Monte_carlo_analysis.py

```
"""
Monte-Carlo statistical mesures of reconstruction
"""
import os
from itertools import product
import time
from multiprocessing import cpu_count
from multiprocessing import Pool
from typing import Callable

import numpy as np

from tomography import random_discrete_image
from tomography.tomographic_reconstruction import radon_transform
from tomography.utils import normalize


# Importance du nombre d'itérations
# Importance nu nombre de prises de vue en angle
# 360° faire de 10 en 10 ?

# La grande mesure de ce TIPE est : Moins d'angle que possible pour le plus de qualité de reconstruction

# [ 10.,  35.,  60.,  85., 110., 135., 160., 185., 210., 235., 260.,
#        285., 310., 335., 360.]

def error_mesure(original: np.ndarray, predicted: np.ndarray) -> float:
    """
    This function mesure the error between the orginal image and the reconstructed image.
    :param original
    :param predicted
    """
    return np.average(np.abs(original - predicted))


def monte_carlo_error_mesure(
        n_mesures: int,
        angles: np.ndarray,
        image_generator: Callable[[], np.ndarray],
        reconstruction: Callable[[np.ndarray, np.ndarray], np.ndarray]
) -> list[float]:
```

```
    """
    This function makes 'n_mesures' simulation of monte-carlo of sinogram reconstruction
    :param n_mesures: Amount of monte-carlo mesure to make
    :param angles: list of angles to capture
    :param image_generator: function that generate a scaner image
    :param reconstruction: a function that reconstruct the image based on sinogram and angle array
    :return: List of errors of each reconstruction
    """
    res = []

    for _ in range(n_mesures):
        image = image_generator()

        sinogram = radon_transform(image, angles)
        recon = reconstruction(sinogram, angles)
        err = error_mesure(normalize(image), normalize(recon))

        res.append(err)

    return res


def _worker_generate(args):
    k, angles = args
    image = random_discrete_image(100, 100, k)
    sinogram = radon_transform(image, angles)
    return image, sinogram.T


def generate_dataset(angles: np.ndarray,
                     amount: int,
                     file_path: str,
                     file_name: str,
                     k: int) -> None:
    params = [(k, angles) for _ in range(amount)]

    with Pool(cpu_count() - 1) as pool:
        results = pool.map(_worker_generate, params)

    data_images, data_sinograms = zip(*results)
```

```
    np.savez(
        file_path + file_name,
        x=np.array(data_sinograms, dtype=np.float32),
        y=np.array(data_images, dtype=np.float32)
    )


if __name__ == '__main__':
    """
    Create the test dataset
    """

    """parameters = [*product(
        [10, 35, 60, 110, 160, 210, 260, 310, 360],
        range(5, 25 + 1, 5),
        range(1)
    )]"""


    # Create the train dataset
    name = "training"
    parameters = [*product(
        [10, 35, 60, 110, 160, 210, 260, 310, 360],
        range(5, 25 + 1, 1),
        range(10)
    )]


    total_length = len(parameters)

    average_wait = []
    count = 0

    for angl, k, i in parameters:
        print(f"Objects: {k}\tID: {i}", end="\t")

        t = time.time()

        generate_dataset(
            np.linspace(0, 360, angl),
            amount=1000,
```

```
            file_path=f"C:/Users/louis/Documents/Python/ADVANCED/Tomographie/data/{name}/",
            file_name=f"data-{k}-{i}-{angl}",
            k=k
        )

        wait = time.time() - t
        average_wait.append(wait)
        count += 1

        print(f"Ended in {wait:.2f} seconds", end="\t")
        print(f"Remaining time: {np.average(average_wait) * (total_length - count):.2f} seconds")

    os.system("shutdown /s /t 60")

"""if __name__== '__main__':

    from __init__import *
    from tomographic_reconstruction import *

    import matplotlib.pyplot as plt
    import matplotlib

    matplotlib.use("TkAgg")

    img_gen = lambda: random_discrete_image(100, 100, 10)

    from tomography.neural_networks import TomographyLSTM
    import keras

    MODEL = TomographyLSTM()
    MODEL.model = keras.models.load_model(
        "C:/Users/louis/Documents/Python/ADVANCED/Tomographie/tomography/models/model_900.keras"
    )

    img_recon = lambda s, a: neural_network_reconstruction_many(s, a, MODEL)

    IMG1 = random_discrete_image(100, 100, 25)
    IMG2 = random_discrete_image(100, 100, 25)

    SINO1 = radon_transform(IMG1, np.linspace(0, 360, 360))
    SINO2 = radon_transform(IMG2, np.linspace(0, 360, 360))
```

```
    RECON1 = neural_network_reconstruction(np.random.uniform(0, 1, size=SINO1.shape), np.linspace(0, 360,
360), MODEL)
    RECON2 = neural_network_reconstruction(SINO2, np.linspace(0, 360, 360), MODEL)

    plt.matshow(RECON1)
    plt.show()
    plt.matshow(RECON2)
    plt.show()

    plt.matshow(RECON1 - RECON2, vmin=-1, vmax=1)
    plt.show()


    IMG = random_discrete_image(100, 100, 10)

    plt.matshow(IMG)
    plt.show()

    SINO = radon_transform(IMG, np.linspace(0, 360, 360))

    RECON = neural_network_reconstruction(SINO, np.linspace(0, 360, 360), MODEL)
    plt.matshow(RECON)
    plt.show()


    plt.matshow(np.abs(normalize(IMG) - normalize(RECON)), vmin=0, vmax=1)
    plt.show()

    '''l = monte_carlo_error_mesure(
        100,
        np.linspace(0, 360, 10_000),
        img_gen,
        img_recon)

    plt.hist(l, bins=51)

    plt.xlim((0, 1))
    plt.show()'''"""

"""if __name__== "__main__":
```

```
    if __name__== '__main__':
        from __init__import *
        from tomographic_reconstruction import *

        import matplotlib.pyplot as plt
        import matplotlib

        matplotlib.use("TkAgg")

        IMG = random_discrete_image(100, 100, 10)


        plt.imshow(IMG)
        plt.show()
"""
```

tomography/Tomograhpic_reconstruction.py

```
"""
This Python libraries contains every function to make a tomography

Fourier-domain reconstruction algorithm
Back projection algorithm
Iterative reconstruction algorithm
"""
import numpy as np
from scipy.ndimage import rotate

from tomography import radon_transform


def back_projection(sinogram: np.ndarray, angles: np.ndarray) -> np.ndarray:
    """
    This function realize the back projection transformation on a sinogram.
    :param sinogram: A sinogram
    :param angles: angles of each view in the sinogram
    :return: Reconstructed image
    """
    n = sinogram.shape[0]
    recon = np.zeros((n, n))

    for i, angle in enumerate(angles):
        proj = sinogram[:, i]

        back_proj = np.tile(proj, (n, 1))
        back_proj_rot = rotate(back_proj, -angle, reshape=False, order=1)
        recon += back_proj_rot

    recon /= len(angles)
    return recon


def fourier_domain_reconstruction(sinogram: np.ndarray, angles: np.ndarray) -> np.ndarray:
    """
    This function realize the fourier domain reconstruction algorithm on a sinogram.
    :param sinogram: A sinogram
    :param angles: angles of each view in the sinogram
    :return: Reconstructed image
    """
```

```
    n = sinogram.shape[0]
    recon = np.zeros((n, n))
    freq = np.fft.fftfreq(n).reshape(-1, 1)

    for i, angle in enumerate(angles):
        proj = sinogram[:, i]
        proj_fft = np.fft.fft(proj)
        ramp = np.abs(freq).ravel()
        proj_fft_filtered = proj_fft * ramp
        proj = np.real(np.fft.ifft(proj_fft_filtered))

        back_proj = np.tile(proj, (n, 1))
        back_proj_rot = rotate(back_proj, -angle, reshape=False, order=1)
        recon += back_proj_rot

    recon /= len(angles)
    return recon


def iterative_reconstruction(sinogram: np.ndarray,
                             angles: np.ndarray,
                             n_iter: int = 10,
                             relaxation: float = 0.1) -> np.ndarray:
    """
    Iterative reconstruction with stability and normalization.
    """
    n = sinogram.shape[0]
    recon = np.zeros((n, n), dtype=np.float32)

    for it in range(n_iter):
        for i, angle in enumerate(angles):
            # Forward projection
            proj_current = radon_transform(recon, [angle])[:, 0]

            # Normalisation to avoid exploding values
            if np.max(proj_current) > 0:
                proj_current /= np.max(proj_current)
            target = sinogram[:, i]
            if np.max(target) > 0:
                target /= np.max(target)
```

```
            # Compute error
            error = target - proj_current

            # Back-propagate correction
            correction = np.tile(error, (n, 1))
            corr_rot = rotate(correction, -angle, reshape=False, order=1)

            # Update with relaxation factor
            recon += relaxation * corr_rot / len(angles)

        # Clip and sanitize to avoid NaNs
        recon = np.clip(recon, 0, None)
        recon = np.nan_to_num(recon, nan=0.0, posinf=0.0, neginf=0.0)

    return recon


def neural_network_reconstruction(sinogram: np.ndarray,
                                  angles: np.ndarray,
                                  model,
                                  *args,
                                  **kwargs) -> np.ndarray:
    new_angle = np.reshape(angles, (len(angles), 1))

    return model(
        sinogram=np.array([sinogram.T]),
        angles=np.array([new_angle]),
        large_input=False,
        *args,
        **kwargs
    )[0]


def neural_network_reconstruction_many(sinograms: np.ndarray,
                                       angles: np.ndarray,
                                       model,
                                       *args,
                                       **kwargs) -> np.ndarray:
    new_angle = np.reshape(angles, (len(angles), 1))
```

```
return model(
    sinogram=np.array([s for s in sinograms]),
    angles=np.array([new_angle for _ in sinograms]),
    large_input=True,
    *args,
    **kwargs
)
```

tomography/utils.py

```python
from copy import deepcopy
from functools import lru_cache

import numpy as np


@lru_cache(maxsize=5)
def limit_circle(height: int, width: int) -> np.ndarray:
    x = np.linspace(-1, 1, width)
    y = np.linspace(-1, 1, height)

    mg_x, mg_y = np.meshgrid(x, y)
    lc = (mg_x ** 2 + mg_y ** 2) <= 1

    return lc

def normalize(arr: np.ndarray, remove_border: bool = True) -> np.ndarray:
    if remove_border:
        val = arr[limit_circle(arr.shape[0], arr.shape[1])]
    else:
        val = arr

    return (arr - np.min(val)) / (np.max(val) - np.min(val))
```

results/1 illustrate problem.ipynb

```
# %% [markdown]
# # Illustrer la tomographie
# %%
import random
from copy import deepcopy

import numpy as np
import plotly.graph_objects as go
import plotly.io as pio
from scipy.ndimage import convolve

from tomography import random_discrete_image
from tomography import random_discrete_image_3d
from tomography.tomographic_reconstruction import radon_transform

pio.renderers.default = "browser"   # plotly_mimetype // browser

COLORSCALE: str = "Viridis"
# %% [markdown]
# ## Illustration 3D
#
# ### Illustration d'un organe fictif
#
# %%
S = 40  # 40
np.random.seed(S)
random.seed(S)

_N = 75
V = random_discrete_image_3d(_N, _N, _N, 20)
CS = 5
kernel = np.ones((CS, CS, CS), dtype=float) / (CS ** 3)
V = convolve(V, kernel, mode='reflect')

X, Y, Z = np.meshgrid(np.linspace(-1, 1, _N), np.linspace(-1, 1, _N), np.linspace(-1, 1, _N))

fig = go.Figure(data=go.Volume(
    x=X.flatten(), y=Y.flatten(), z=Z.flatten(),
    value=V.flatten(),
    isomin=0.1,
    isomax=1.0,
```

```
    opacity=0.15,
    surface_count=5,
    colorscale=COLORSCALE,
    showscale=False,
))


fig.update_yaxes(showticklabels=False, visible=False)

fig.update_layout(
    template='plotly_white',
    scene_camera=dict(
        up=dict(x=0, y=0, z=-1),
        center=dict(x=-0, y=0, z=-0.1),
        eye=dict(x=1.2, y=1.2, z=1.2)
    ),
    margin=dict(
        l=0,
        r=0,
        b=0,
        t=0,
    ),
    autosize=True,
    scene=dict(
        xaxis=dict(
            showticklabels=False,
            title={"text": ""},
            range=[-1, 1],
            tick0=0,
            dtick=0.2,
        ),
        yaxis=dict(
            showticklabels=False,
            title={"text": ""},
            range=[-1, 1],
            tick0=0,
            dtick=0.2,
        ),
        zaxis=dict(
            showticklabels=False,
            title={"text": ""},
```

```
            range=[-1, 1],
            tick0=0,
            dtick=0.2,
        ),
    )
)

# fig.show()

fig.write_image("images/1 fig0 lr.webp", scale=1.3 * 2, height=600, width=600)
fig.write_image("images/1 fig0 hr.png", scale=13, height=600, width=600)
# %% [markdown]
# ### Illustration de ce que va faire la caméra
# %%
cx, cy, cz = 0, 0, 0    # cercle sur le plan z=0
r = 1 # rayon du cercle

theta = np.linspace(0, 2 * np.pi, 200)
x_circle = cx + r * np.cos(theta)
y_circle = cy + r * np.sin(theta)
z_circle = np.full_like(x_circle, cz)

fig.add_trace(go.Scatter3d(
    x=x_circle,
    y=y_circle,
    z=z_circle,
    mode='lines',
    line=dict(color='black', width=7.5),
    name='Cercle',
    showlegend=False
))

# fig.show()

fig.write_image("images/1 fig1 lr.webp", scale=1.3 * 2, height=600, width=600)
fig.write_image("images/1 fig1 hr.png", scale=13 / 4, height=600 * 4, width=600 * 4)

# %% [markdown]
# ## Idée générale
#
# ### Illustration fictive de densité osseuse
```

```
# %%

S = 65  # 55 65
np.random.seed(S)
random.seed(S)

image = random_discrete_image(1000, 1000, 20)

empty_img = deepcopy(image)
empty_img[empty_img == 0] = np.nan

fig = go.Figure(data=go.Heatmap(
    z=empty_img,
    colorscale=COLORSCALE,
    colorbar=dict(
        title="Densité",
    ),
)
)

fig.update_xaxes(scaleanchor='x',
                 linewidth=1,
                 linecolor='black',
                 mirror=True,
                 ticks="",
                 showline=True,
                 showgrid=False,
                 scaleratio=1,
                 showticklabels=False)
fig.update_yaxes(scaleanchor='y',
                 linewidth=1,
                 linecolor='black',
                 mirror=True,
                 ticks="",
                 showline=True,
                 showgrid=False,
                 scaleratio=1,
                 showticklabels=False,
                 )

fig.update_layout(
```

```
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=0.99,
        xanchor="left",
        x=0.025,
        bordercolor="black",
        borderwidth=1),
    title={"text": r"$\LARGE{\text{Simulation d'un densité osseuse}}$"},
)

# fig.show()

fig.write_image("images/1 fig2 lr.webp", scale=1.3 * 2, height=600, width=600)
fig.write_image("images/1 fig2 hr.png", scale=13, height=600, width=600)
# %% [markdown]
# ### Illustration du sinogramme
# %%
rad = radon_transform(image, np.linspace(0, 360, 1000))

fig = go.Figure(
    data=go.Heatmap(
        x=np.linspace(0, 360, 1000),
        z=rad,
        colorscale=COLORSCALE,
        showscale=False,
    )
)

fig.update_xaxes(scaleanchor='x',
                 linewidth=1,
                 linecolor='black',
                 mirror=True,
                 ticks='outside',
                 showline=True,
                 showgrid=False,
                 scaleratio=1)
fig.update_yaxes(scaleanchor='y',
                 linewidth=1,
                 linecolor='black',
                 mirror=True,
```

```
                  ticks=None,
                  showline=True,
                  showgrid=False,
                  scaleratio=1,
                  showticklabels=False,)

fig.update_layout(
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=0.99,
        xanchor="left",
        x=0.025,
        bordercolor="black",
        borderwidth=1),
    title={"text": r"$\LARGE{\text{Sinogramme de la densité osseuse}}$"},
    xaxis={
        "title": {"text": r"$\Large{\text{Angles de vu en degré }}$"},
        "tickvals": np.linspace(0, 360, 10),
        "ticktext": [f"{int(v)}°" for v in np.linspace(0, 360, 10)]
    }
)

# fig.show()

fig.write_image("images/1 fig3 lr.webp", scale=1.3 * 2, height=600, width=600)
fig.write_image("images/1 fig3 hr.png", scale=13, height=600, width=600)
```

results/2
iterative_reconstruction_parameter.py

```python
"""
Influence du nombre d'itérations pour la méthode de reconstruction itérative
"""
from multiprocessing import Pool
import sqlite3

import numpy as np

from tomography import random_discrete_image
from tomography.monte_carlo_analysis import monte_carlo_error_mesure
from tomography.tomographic_reconstruction import iterative_reconstruction

# parameters
IMG_SIZE: int = 100
ELLIPSIS_COUNT: int = 15
N_PROCESS: int = 22
N_ITER_MAX: int = 20

MESURE_COUNT: int = 1000
BATCH_SIZE: int = 25

img_gen = lambda: random_discrete_image(IMG_SIZE, IMG_SIZE, ELLIPSIS_COUNT)

# open/init the database
conn = sqlite3.connect(f"data/iterative_parameters.db", check_same_thread=False)
cursor = conn.cursor()

try:
    cursor.execute("""CREATE TABLE "results" ("n_iter" INTEGER NOT NULL, "error" REAL NOT NULL);""")
except Exception as e:
    print(e)


def mesure_process(param: tuple[int, int, int, int]) -> None:
    """
    Process to store the error of the iterative reconstruction in the database
    :param param: All parameters : id, batch_id, batch_size, n_iter
    :return: None
    """
    _id, batch_num, batch_size, n_iter = param
```

```
    adv = round(_id / (MESURE_COUNT * N_ITER_MAX) * BATCH_SIZE * 100, 2)
    print(f"Starting {_id}: {batch_num} {batch_size} {n_iter} ({adv}%)")

    img_recon = lambda s, a: iterative_reconstruction(s, a, n_iter=n_iter)

    res: list[float] = monte_carlo_error_mesure(
        batch_size,
        np.linspace(0, 360, 360),
        img_gen,
        img_recon
    )

    for val in res:
        conn.execute("INSERT INTO results (n_iter, error) VALUES (?, ?)", (n_iter, val))

    conn.commit()


if __name__ == '__main__':
    task = [(bnum, BATCH_SIZE, k) for k in range(1, N_ITER_MAX + 1) for bnum in range(MESURE_COUNT //
BATCH_SIZE)]
    task = [*enumerate(task)]
    task = [*map(lambda c: (c[0],) + c[1], task)]

    with Pool(processes=N_PROCESS) as pool:
        map_result = pool.imap_unordered(mesure_process, [*task])
        for r in map_result:
            continue
```

results/2 mathematical methods.ipynb

```
# %% [markdown]
# # Méthodes mathématiques de résolution
# %%
from copy import deepcopy
import random
import sqlite3

import numpy as np
import plotly.graph_objects as go
import plotly.io as pio

from tomography import random_discrete_image
from tomography.tomographic_reconstruction import back_projection
from tomography.tomographic_reconstruction import fourier_domain_reconstruction
from tomography.tomographic_reconstruction import iterative_reconstruction
from tomography.tomographic_reconstruction import radon_transform

pio.renderers.default = "browser"   # plotly_mimetype // browser

COLORSCALE: str = "Viridis"
# %%
S = 65  # 55 65
np.random.seed(S)
random.seed(S)

image = random_discrete_image(100, 100, 20)

empty_img = deepcopy(image)
empty_img[empty_img == 0] = np.nan

fig = go.Figure(data=go.Heatmap(
    z=empty_img,
    colorscale=COLORSCALE,
    showscale=False,
)
)

fig.update_xaxes(scaleanchor='x',
                 linewidth=1,
                 linecolor='black',
                 mirror=True,
```

```
                  ticks='outside',
                  showline=True,
                  showgrid=False,
                  scaleratio=1,
                  showticklabels=False)
fig.update_yaxes(scaleanchor='y',
                  linewidth=1,
                  linecolor='black',
                  mirror=True,
                  ticks="outside",
                  showline=True,
                  showgrid=False,
                  scaleratio=1,
                  showticklabels=False,
                  )

fig.update_layout(
    template='plotly_white',
    legend=dict(
        yanchor="top",
        y=0.99,
        xanchor="left",
        x=0.025,
        bordercolor="black",
        borderwidth=1),
    title={"text": r"$\LARGE{\text{Simulation d'un densité osseuse}}$"},
)

# fig.write_image("images/2 fig0 lr.webp", scale=1.3 * 2, height=600, width=600)
# fig.write_image("images/2 fig0 hr.png", scale=13, height=600, width=600)
# %% [markdown]
# ## Les méthodes
#
# ### Back Projection
# %%
for i, n_angle in enumerate([10, 25, 100, 360]):
    print(n_angle)
    angles = np.linspace(0, 360, n_angle)
    sinogram = radon_transform(image, angles)

    reconstruction = back_projection(sinogram, angles)
```

```
    fig = go.Figure(data=go.Heatmap(
        z=reconstruction,
        colorscale=COLORSCALE,
        showscale=False,
    ))

    fig.update_layout(
        template='plotly_white',
        title={
            "text": r"$\LARGE{\text{Reconstruction de la densité osseuse}} \\ \large{\text{Rétroprojection
- " + f"{n_angle} prises de vues" + "}} $"
        },
    )

    fig.update_xaxes(scaleanchor='x',
                     linewidth=1,
                     linecolor='black',
                     scaleratio=1,
                     showticklabels=False,
                     ticks=None, )
    fig.update_yaxes(scaleanchor='y',
                     linewidth=1,
                     linecolor='black',
                     showticklabels=False,
                     ticks=None,
                     )

    fig.write_image(f"images/2 fig1-{i} lr.webp", scale=1.3, height=600, width=600)
    fig.write_image(f"images/2 fig1-{i} hr.png", scale=13, height=600, width=600)

# %% [markdown]
# ### Fourier domain reconstruction
# %%
for i, n_angle in enumerate([10, 25, 100, 360]):
    print(n_angle)
    angles = np.linspace(0, 360, n_angle)
    sinogram = radon_transform(image, angles)

    reconstruction = fourier_domain_reconstruction(sinogram, angles)
```

```
    fig = go.Figure(data=go.Heatmap(
        z=reconstruction,
        colorscale=COLORSCALE,
        showscale=False,
    ))

    fig.update_layout(
        template='plotly_white',
        title={
            "text": r"$\LARGE{\text{Reconstruction de la densité osseuse}} \\ \large{\text{Reconstruction
dans le domaine de Fourier - " + f"{n_angle} prises de vues" + "}} $"
        },
    )

    fig.update_xaxes(scaleanchor='x',
                     linewidth=1,
                     linecolor='black',
                     scaleratio=1,
                     showticklabels=False,
                     ticks=None, )
    fig.update_yaxes(scaleanchor='y',
                     linewidth=1,
                     linecolor='black',
                     showticklabels=False,
                     ticks=None,
                     )

    fig.write_image(f"images/2 fig2-{i} lr.webp", scale=1.3, height=600, width=600)
    fig.write_image(f"images/2 fig2-{i} hr.png", scale=13, height=600, width=600)

# %% [markdown]
# ### Iterative reconstruction
# %%
for i, n_angle in enumerate([10, 25, 100, 360]):
    print(n_angle)
    angles = np.linspace(0, 360, n_angle)
    sinogram = radon_transform(image, angles)

    reconstruction = iterative_reconstruction(sinogram, angles, n_iter=2)

    fig = go.Figure(data=go.Heatmap(
```

```
        z=reconstruction,
        colorscale=COLORSCALE,
        showscale=False,
    ))

    fig.update_layout(
        template='plotly_white',
        title={
            "text": r"$\LARGE{\text{Reconstruction de la densité osseuse}} \\ \large{\text{Reconstruction
itérative (5 itérations) - " + f"{n_angle} prises de vues" + "}} $"
        },
    )

    fig.update_xaxes(scaleanchor='x',
                     linewidth=1,
                     linecolor='black',
                     scaleratio=1,
                     showticklabels=False,
                     ticks=None, )
    fig.update_yaxes(scaleanchor='y',
                     linewidth=1,
                     linecolor='black',
                     showticklabels=False,
                     ticks=None,
                     )

    fig.write_image(f"images/2 fig3-{i} lr.webp", scale=1.3, height=600, width=600)
    fig.write_image(f"images/2 fig3-{i} hr.png", scale=13, height=600, width=600)

# %% [markdown]
# ## Nombre d'itérations pour la reconstruction interactive
# %%
conn = sqlite3.connect(f"data/iterative_parameters.db", check_same_thread=False)
cursor = conn.cursor()

data = [*cursor.execute("SELECT * FROM results").fetchall()]

cursor.close()
conn.close()

x = np.array([*map(lambda c: c[0], data)])
```

```
y = np.array([*map(lambda c: c[1], data)])
# %%
# Calculs statistiques avec NumPy
x_unique = np.arange(1, max(x) + 1)

# Listes des statistiques par catégorie
q1, q2, q3 = [], [], []
lowerfence, upperfence = [], []
mean, sd = [], []

# Calcul des quartiles et bornes par groupe de x
for xv in x_unique:
    y_group = y[x == xv]
    if len(y_group) == 0:
        q1.append(np.nan)
        q2.append(np.nan)
        q3.append(np.nan)
        lowerfence.append(np.nan)
        upperfence.append(np.nan)
        mean.append(np.nan)
        sd.append(np.nan)
        continue

    q1v = np.percentile(y_group, 25)
    q2v = np.percentile(y_group, 50)
    q3v = np.percentile(y_group, 75)
    iqr = q3v - q1v

    # Moustaches limitées par les valeurs réelles
    lower = max(np.min(y_group), q1v - 1.5 * iqr)
    upper = min(np.max(y_group), q3v + 1.5 * iqr)

    q1.append(q1v)
    q2.append(q2v)
    q3.append(q3v)
    lowerfence.append(lower)
    upperfence.append(upper)
    mean.append(np.mean(y_group))
    sd.append(np.std(y_group))

fig = go.Figure()
```

```
fig.add_trace(go.Box(
    x=x_unique,
    q1=q1,
    median=q2,
    q3=q3,
    lowerfence=lowerfence,
    upperfence=upperfence,
    mean=mean,
    sd=sd,
    boxmean='sd',
    line=dict(width=1.75),
))

fig.update_layout(
    template='plotly_white',
    title={
        "text": r"$\LARGE{\text{Comparaison de l'erreur en fonction du nombre d'itérations}} \\
\large{\text{Méthode itérative - 10 000 mesures par boîte - 360 prises de vues}}$"
    },
)

fig.update_xaxes(scaleanchor='x',
                 linewidth=1,
                 linecolor='black',
                 scaleratio=1,
                 showticklabels=True,
                 dtick=1,
                 )
fig.update_yaxes(scaleanchor='y',
                 linewidth=1,
                 linecolor='black',
                 showticklabels=True,
                 range=[0.0, 0.3]
                 )

fig.write_image(f"images/2 fig4 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image(f"images/2 fig4 hr.png", scale=13, height=600, width=1038)
# %%
fig = go.Figure()
```

```
fig.add_trace(go.Scatter(
    x=x_unique,
    y=mean,
    mode='lines+markers',
    line=dict(width=2, color='royalblue'),
    marker=dict(size=7, symbol='circle', color='royalblue')
))

fig.update_layout(
    template='plotly_white',
    title={
        "text": r"$\LARGE{\text{Comparaison de l'erreur moyenne en fonction du nombre d'itérations}} \\
\large{\text{Méthode itérative - 10 000 mesures par point - 360 prises de vues}}$"
    },
)

fig.update_xaxes(scaleanchor='x',
                 linewidth=1,
                 linecolor='black',
                 scaleratio=1,
                 showticklabels=True,
                 dtick=1,
                 )
fig.update_yaxes(scaleanchor='y',
                 linewidth=1,
                 linecolor='black',
                 showticklabels=True,
                 range=[0.0, 0.15]
                 )

fig.write_image(f"images/2 fig5 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image(f"images/2 fig5 hr.png", scale=13, height=600, width=1038)
# %%

# %% [markdown]
# ## NN
#
# %%
import numpy as np

__import__("os").environ["CUDA_VISIBLE_DEVICES"] = ""
```

```
from tomography.neural_networks.direct_reconstruction import CTReconstructionNetwork

MODEL_DIRECT = CTReconstructionNetwork.load(

"C:/Users/louis/Documents/Python/ADVANCED/Tomographie/tomography/neural_networks/models/Direct_0011.keras")
MODEL_DIRECT.projection_dim = 128
MODEL_DIRECT.latent_dim = 256
# %%
for i, n_angle in enumerate([10, 25, 100, 360]):
    angles = np.linspace(0, 360, n_angle)
    sinogram = radon_transform(image, angles)
    print(sinogram.shape)
    reconstruction = MODEL_DIRECT.predict(sinogram.T, angles_deg=angles)

    fig = go.Figure(data=go.Heatmap(
        z=reconstruction,
        colorscale=COLORSCALE,
        showscale=False,
    ))

    fig.update_layout(
        template='plotly_white',
        title={
            "text": r"$\LARGE{\text{Reconstruction de la densité osseuse}} \\ \large{\text{Réseau de
neurone - " + f"{n_angle} prises de vues" + "}} $"
        },
    )

    fig.update_xaxes(scaleanchor='x',
                     linewidth=1,
                     linecolor='black',
                     scaleratio=1,
                     showticklabels=False,
                     ticks=None, )
    fig.update_yaxes(scaleanchor='y',
                     linewidth=1,
                     linecolor='black',
                     showticklabels=False,
                     ticks=None,
```

```
                    )

    fig.write_image(f"images/5 fig5-{i} lr.webp", scale=1.3, height=600, width=600)
    fig.write_image(f"images/5 fig5-{i} hr.png", scale=13, height=600, width=600)

# %%
from tomography.neural_networks.unet import CTUNet

MODEL = CTUNet.load(

"C:/Users/louis/Documents/Python/ADVANCED/Tomographie/tomography/neural_networks/models/UNet_fft_0012.keras
")
MODEL.bp_type = "fft"

for i, n_angle in enumerate([10, 25, 100, 360]):
    angles = np.linspace(0, 360, n_angle)
    sinogram = radon_transform(image, angles)
    print(sinogram.shape)
    reconstruction = MODEL.predict(sinogram.T, angles_deg=angles)

    fig = go.Figure(data=go.Heatmap(
        z=reconstruction,
        colorscale=COLORSCALE,
        showscale=False,
    ))

    fig.update_layout(
        template='plotly_white',
        title={
            "text": r"$\LARGE{\text{Reconstruction de la densité osseuse}} \\ \large{\text{Réseau de
neurone - " + f"{n_angle} prises de vues" + "}} $"
        },
    )

    fig.update_xaxes(scaleanchor='x',
                     linewidth=1,
                     linecolor='black',
                     scaleratio=1,
                     showticklabels=False,
                     ticks=None, )
    fig.update_yaxes(scaleanchor='y',
```

```
                    linewidth=1,
                    linecolor='black',
                    showticklabels=False,
                    ticks=None,
                    )

fig.write_image(f"images/5 fig6-{i} lr.webp", scale=1.3, height=600, width=600)
fig.write_image(f"images/5 fig6-{i} hr.png", scale=13, height=600, width=600)
```

results/3 methods_comparison.py

```
# %% [markdown]
# # Comparaison des méthodes mathématiques
#
# %%
from copy import deepcopy
import random
import sqlite3

import numpy as np
import plotly.graph_objects as go

METHODS = {
    "bp": "Backprojection",
    "fd": "Filtered BP",
    "ir": "Iterative",
    "nn": "Réseau de neurones",
    "unet_bp": "UNet BP",
    "unet_fft": "UNet FFT",
}
# %% [markdown]
# ## Erreur par rapport au nombre d'objets
# %%


conn = sqlite3.connect("data/methods_comparison.db")
cursor = conn.cursor()

x = []
y = []

for table in ["bp", "fd", "ir"]:
    cursor.execute(f"""
        SELECT n_shape, error
        FROM results_{table}
    """)
    for n_shape, err in cursor.fetchall():
        x.append(str(n_shape))
        y.append(max(0, err))

conn.close()
# %%
```

```
fig = go.Figure()

fig.add_trace(
    go.Box(
        x=x,
        y=y,
        notched=True,
        boxpoints=False,
        boxmean=True,
        line=dict(width=1.75),
        showlegend=False
    )
)

fig.update_layout(
    template='plotly_white',
    title={
        "text": r"$\Large{\text{Erreur de reconstruction (toutes méthodes confondues) en fonction du nombre
d'objets}}  \\ \large{\text{10 000 mesures par point - toutes prises de vues}}$"
    },
    xaxis={
        "title": {"text": r"$\Large{\text{Nombre d'objets}}$"},
        "type": "category"
    },
    yaxis={"title": {"text": r"$\Large{\text{Erreur}}$"}},
)

fig.write_image("images/3 fig1 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image("images/3 fig1 hr.png", scale=10, height=600, width=1038)
# %%
from collections import defaultdict

stats = defaultdict(lambda: defaultdict(list))

conn = sqlite3.connect("data/methods_comparison.db")
cursor = conn.cursor()

for key in ["bp", "fd", "ir"]:
    cursor.execute(f"""
        SELECT n_angle, error
        FROM results_{key}
```

```
    """)
    for n_angle, err in cursor.fetchall():
        stats[key][n_angle].append(max(0, err))

conn.close()
# %%
fig = go.Figure()

for key, method_name in METHODS.items():
    angles = sorted(stats[key].keys())

    means = sorted([np.mean(stats[key][a]) for a in angles], reverse=True)
    stds  = [np.std(stats[key][a]) for a in angles]
    ns    = [len(stats[key][a]) for a in angles]

    fig.add_trace(
        go.Scatter(
            x=angles,
            y=means,
            mode='lines+markers',
            name=method_name,
            error_y=dict(
                type='data',
                array=np.array(stds),

                thickness=1.5,
                width=5,
            )
        )
    )
# %% [markdown]
#
# %%
fig.update_layout(
    template='plotly_white',
    title={
        "text": r"$\Large{\text{Influence du nombre d'angles de vue sur l'erreur}} \\ \large{\text{10 000
mesures par point}}$"
                r""
    },
    xaxis={
```

```
        "title": {"text": r"$\Large{\text{Nombre d'angles de vue}}$"},
        "showline": True,
        "mirror": True
    },
    yaxis={
        "title": {"text": r"$\Large{\text{Erreur moyenne}}$"},
        "showline": True,
        "mirror": True
    },
    legend=dict(
        yanchor="top",
        y=0.99,
        xanchor="left",
        x=0.75,
        bordercolor="black",
        borderwidth=1
    )
)

fig.write_image("images/3 fig2 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image("images/3 fig2 hr.png", scale=10, height=600, width=1038)
# %% [markdown]
# ## Matrice de corrélation
# %%
# Chargement des données
conn = sqlite3.connect("data/methods_comparison.db")
cursor = conn.cursor()

all_series = []

for method in ["bp", "fd", "ir"]:
    cursor.execute(f"SELECT error FROM results_{method} ORDER BY n_shape ASC, n_angle ASC, id ASC, sub_id
ASC;")
    all_series.append(
        np.array([max(0, e) for (e,) in cursor.fetchall()], dtype=float)
    )

conn.close()

# Matrice de corrélation
N = len(["bp", "fd", "ir"])
```

```
corr_matrix = np.ones((N, N))

for i in range(N):
    for j in range(i + 1, N):

        mask = ~np.isnan(all_series[i]) & ~np.isnan(all_series[j])

        si = all_series[i][mask]
        sj = all_series[j][mask]

        corr = np.mean((si - si.mean()) * (sj - sj.mean())) / (si.std() * sj.std())

        corr_matrix[i, j] = corr
        corr_matrix[j, i] = corr

# Heatmap
fig = go.Figure(go.Heatmap(
    z=corr_matrix,
    x=[m.upper() for m in ["bp", "fd", "ir"]],
    y=[m.upper() for m in ["bp", "fd", "ir"]],
    zmin=-1,
    zmax=1,
    colorscale="RdYlGn",
    text=np.round(corr_matrix, 2),
    texttemplate="%{text}"
))

fig.update_yaxes(autorange="reversed")

fig.update_layout(
    template="plotly_white",

    title={
        "text":
            r"$\Large{\text{Matrice de corrélation des erreurs}}$"
            r"<br>"
            r"$\large{\text{Méthodes BP / FD / IR}}$"
    },

    xaxis={
        "title": {"text": r"$\Large{\text{Méthode}}$"},
```

```
        "side": "bottom"
    },

    yaxis={
        "title": {"text": r"$\Large{\text{Méthode}}$"}
    },

    width=700,
    height=700
)

fig.write_image("images/3 fig3 tri lr.webp", scale=1.3 * 2, height=600, width=600)
fig.write_image("images/3 fig3 tri hr.png", scale=13, height=600, width=600)
# %% [markdown]
# ## Test avec NN
# %%
conn = sqlite3.connect("data/methods_comparison.db")
cursor = conn.cursor()

x = []
y = []

for table, alpha in zip(["ir", "unet_bp", "unet_fft", "nn"], [1, 1, 1, 1]):
    cursor.execute(f"""
        SELECT n_shape, error
        FROM results_{table}
    """)
    for n_shape, err in cursor.fetchall():
        x.append(str(n_shape))
        y.append(max(0, err) / alpha)

conn.close()
# %%
from collections import defaultdict

stats = defaultdict(lambda: defaultdict(list))

conn = sqlite3.connect("data/methods_comparison.db")
cursor = conn.cursor()

for key in [k for k in METHODS.keys() if k != "bp" and k != "fd"]:
```

```
    cursor.execute(f"""
        SELECT n_angle, error
        FROM results_{key}
    """)
    for n_angle, err in cursor.fetchall():
        stats[key][n_angle].append(max(0, err))

conn.close()
# %%
fig = go.Figure()

for key, alpha in zip(["ir", "unet_bp", "unet_fft", "nn"], [1, 1, 1, 1]):
    method_name = METHODS[key]
    angles = sorted(stats[key].keys())

    means = sorted([np.mean(stats[key][a]) / alpha for a in angles], reverse=True)
    stds  = [np.std(stats[key][a]) for a in angles]
    ns    = [len(stats[key][a]) for a in angles]

    fig.add_trace(
        go.Scatter(
            x=angles,
            y=means,
            mode='lines+markers',
            name=method_name,
        )
    )



fig.update_layout(
    template='plotly_white',
    title={
        "text": r"$\Large{\text{Influence du nombre d'angles de vue sur l'erreur}} \\ \large{\text{10 000
mesures par point}}$"
               r""
    },
    xaxis={
        "title": {"text": r"$\Large{\text{Nombre d'angles de vue}}$"},
        "showline": True,
        "mirror": True
```

```
    },
    yaxis={
        "title": {"text": r"$\Large{\text{Erreur moyenne}}$"},
        "showline": True,
        "mirror": True
    },
    legend=dict(
        yanchor="top",
        y=0.99,
        xanchor="left",
        x=0.75,
        bordercolor="black",
        borderwidth=1
    )
)

fig.write_image("images/5 fig2 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image("images/5 fig2 hr.png", scale=10, height=600, width=1038)
# %%
# Chargement des données
conn = sqlite3.connect("data/methods_comparison.db")
cursor = conn.cursor()

all_series = []

for method, alpha in zip(METHODS.keys(), [1, 1, 2, 5]):
    cursor.execute(f"SELECT error FROM results_{method} ORDER BY n_shape ASC, n_angle ASC, id ASC, sub_id
ASC;")
    all_series.append(
        np.array([max(0, e) / alpha for (e,) in cursor.fetchall()], dtype=float)
    )

conn.close()

# Matrice de corrélation
N = len(METHODS.keys())
corr_matrix = np.ones((N, N))

for i in range(N):
    for j in range(i + 1, N):
```

```
        mask = ~np.isnan(all_series[i]) & ~np.isnan(all_series[j])

        si = all_series[i][mask]
        sj = all_series[j][mask]

        corr = np.mean((si - si.mean()) * (sj - sj.mean())) / (si.std() * sj.std())

        corr_matrix[i, j] = corr
        corr_matrix[j, i] = corr

# Heatmap
fig = go.Figure(go.Heatmap(
    z=corr_matrix,
    x=[m.upper() for m in METHODS.keys()],
    y=[m.upper() for m in METHODS.keys()],
    zmin=-1,
    zmax=1,
    colorscale="RdYlGn",
    text=np.round(corr_matrix, 2),
    texttemplate="%{text}"
))

fig.update_yaxes(autorange="reversed")

fig.update_layout(
    template="plotly_white",

    title={
        "text":
            r"$\Large{\text{Matrice de corrélation des erreurs}}$"
            r"<br>"
            r"$\large{\text{Méthodes BP / FD / IR}}$"
    },

    xaxis={
        "title": {"text": r"$\Large{\text{Méthode}}$"},
        "side": "bottom"
    },

    yaxis={
        "title": {"text": r"$\Large{\text{Méthode}}$"}
```

```
    },

    width=700,
    height=700
)

fig.write_image("images/5 fig3 tri lr.webp", scale=1.3 * 2, height=600, width=600)
fig.write_image("images/5 fig3 tri hr.png", scale=13, height=600, width=600)
# %%
from scipy import stats as stats_scipy

from collections import defaultdict

stats = defaultdict(lambda: defaultdict(list))

conn = sqlite3.connect("data/methods_comparison.db")
cursor = conn.cursor()

for key in [k for k in METHODS.keys() if k != "bp" and k != "fd"]:
    cursor.execute(f"""
        SELECT n_angle, error
        FROM results_{key}
    """)
    for n_angle, err in cursor.fetchall():
        stats[key][n_angle].append(max(0, err))

conn.close()

# from scipy import stats as scipy_stats

for method in ["unet_fft", "unet_bp"]:
    angles = sorted(set(stats["ir"].keys()) & set(stats[method].keys()))

    errors_ir = []
    errors_method = []

    for angle in angles:
        errors_ir.extend(stats["ir"][angle])
        errors_method.extend(stats[method][angle])

    n = min(len(errors_ir), len(errors_method))
```

```
    stat, p_value = stats_scipy.wilcoxon(
        errors_method,
        errors_ir,
        alternative="less"
    )

    print(f"\n{method} vs ir :")
    print(f"  p-value = {p_value:.4f}", " significatif (< 0.05)" if p_value < 0.05 else " non
significatif")
```

results/3 methods_comparison.ipynb

```
"""
Compare all methods
"""
__import__("os").environ["CUDA_VISIBLE_DEVICES"] = ""

import os
import sqlite3
import time
from itertools import product
from multiprocessing import cpu_count
from multiprocessing import Pool

import numpy as np

from tomography.monte_carlo_analysis import error_mesure
from tomography.neural_networks.unet import CTUNet
from tomography.neural_networks.direct_reconstruction import CTReconstructionNetwork
from tomography.tomographic_reconstruction import back_projection
from tomography.tomographic_reconstruction import fourier_domain_reconstruction
from tomography.tomographic_reconstruction import iterative_reconstruction
from tomography.utils import normalize

METHODS_IDS: dict[str, int] = {n: i for i, n in enumerate(["bp", "fd", "ir", "nn", "unet_bp", "unet_fft"])}
METHODS_LABEL: dict[int, str] = {i: n for i, n in enumerate(["bp", "fd", "ir", "nn", "unet_bp",
"unet_fft"])}

#
# TEST_PATH: str = "C:/Users/louis/Documents/Python/ADVANCED/Tomographie/data/testing"
TEST_PATH: str = "/mnt/c/Users/louis/Documents/Python/ADVANCED/Tomographie/data/testing"
MODEL = None

MODEL_BP = CTUNet.load(

"/mnt/c/Users/louis/Documents/Python/ADVANCED/Tomographie/tomography/neural_networks/models/UNet_0010.keras
")
MODEL_BP.bp_type = "bp"

MODEL_FFT = CTUNet.load(

"/mnt/c/Users/louis/Documents/Python/ADVANCED/Tomographie/tomography/neural_networks/models/UNet_fft_0012.k
eras")
```

```
MODEL_FFT.bp_type = "fft"

MODEL_DIRECT = CTReconstructionNetwork.load(

"/mnt/c/Users/louis/Documents/Python/ADVANCED/Tomographie/tomography/neural_networks/models/Direct_0011.ker
as")
MODEL_DIRECT.projection_dim = 128
MODEL_DIRECT.latent_dim = 256


def mesure_process(param: tuple[str, int, int, int]):
    method_label: str = param[0]
    n_shape = param[1]
    n_angle = param[2]
    _id = param[3]

    method_id = METHODS_IDS[method_label]

    print(f"Started: {method_label}\tshapes:{n_shape}\tangles:{n_angle}\tid:{_id}")
    t = time.time()

    data = np.load(os.path.join(TEST_PATH, f"data-{n_shape}-{_id}-{n_angle}.npz"))
    sinograms = data["x"]
    images = data["y"]
    angles = np.linspace(0, 360, n_angle)

    results = []  # (method_label, n_shape, n_angle, id, sub_id, error)

    if method_label == "unet_bp":
        preds = [p for p in MODEL_BP.predict(sinograms, angles_deg=angles)]

        for i, (sin, img, pred) in enumerate(zip(sinograms, images, preds)):
            err = np.average(np.abs(normalize(img) - normalize(pred)))
            results.append((method_label, n_shape, n_angle, _id, i, float(err)))

    elif method_label == "unet_fft":
        preds = [p for p in MODEL_FFT.predict(sinograms, angles_deg=angles)]

        for i, (sin, img, pred) in enumerate(zip(sinograms, images, preds)):
            err = np.average(np.abs(normalize(img) - normalize(pred)))
            results.append((method_label, n_shape, n_angle, _id, i, float(err)))
```

```
    elif method_label == "nn":
        preds = [p for p in[ MODEL_DIRECT.predict(s, angles_deg=angles) for s in sinograms]]

        for i, (sin, img, pred) in enumerate(zip(sinograms, images, preds)):
            err = np.average(np.abs(normalize(img) - normalize(pred)))
            results.append((method_label, n_shape, n_angle, _id, i, float(err)))

    else:
        if method_label == "bp":
            func = back_projection
        elif method_label == "fd":
            func = fourier_domain_reconstruction
        elif method_label == "ir":
            func = lambda s, a: iterative_reconstruction(s, a, 10)
        else:
            raise Exception("Unknown method")

        for i, (sin, img) in enumerate(zip(sinograms, images)):
            pred = func(sin.T, angles)
            err = error_mesure(normalize(img), normalize(pred))
            results.append((method_label, n_shape, n_angle, _id, i, float(err)))

    duration = time.time() - t
    print(f"Ended in {duration:.2f}s: {method_label}\tshapes:{n_shape}\tangles:{n_angle}\tid:{_id}")

    return method_id, n_shape, n_angle, _id, results


if __name__ == '__main__':
    """
    Test basics methods
    """

    # Create database
    try:
        conn = sqlite3.connect("data/methods_comparison.db")
        cursor = conn.cursor()

        # Create all tables for results
```

```
        for name in ["nn", "unet_bp", "unet_fft", "bp", "fd", "ir", ]:
            cursor.execute(f"""CREATE TABLE "results_{name}" (
                "n_shape" INTEGER NOT NULL,
                "n_angle" INTEGER NOT NULL,
                "id" INTEGER NOT NULL,
                "sub_id" INTEGER NOT NULL,
                "error" REAL NOT NULL
            );
            """)

        # Create main table to get the remaining mesure to do
        cursor.execute(f"""CREATE TABLE "total_results" (
                "method_id" INTEGER NOT NULL,
                "n_shape" INTEGER NOT NULL,
                "n_angle" INTEGER NOT NULL,
                "id" INTEGER NOT NULL
            );
            """)

        conn.close()

    except Exception as e:
        print(e)

    params = list(product(
        ["nn"],
        range(5, 25 + 1, 5),
        [10, 35, 60, 110, 160, 210, 260, 310, 360],
        [0]
    ))

    # [10, 35, 60, 110, 160, 210, 260, 310, 360], #   260, 310, 360

    conn = sqlite3.connect("data/methods_comparison.db")
    cursor = conn.cursor()

    '''
    #
    # Multiprocess version :
    #
```

```
with Pool(1) as pool:
    for method_id, n_shape, n_angle, _id, res in pool.imap_unordered(mesure_process, params):

        # Skip if already done
        cursor.execute(
            "SELECT 1 FROM total_results WHERE method_id=? AND n_shape=? AND n_angle=? AND id=?",
            (method_id, n_shape, n_angle, _id)
        )
        if cursor.fetchone():
            print(f"Already done: {method_id} {n_shape} {n_angle} {_id}")
            continue

        # Cleanup partial runs
        method_label = [k for k, v in METHODS_IDS.items() if v == method_id][0]
        cursor.execute(
            f"DELETE FROM results_{method_label} WHERE n_shape=? AND n_angle=? AND id=?",
            (n_shape, n_angle, _id)
        )

        # Bulk insert (rapide)
        cursor.executemany(
            f"""
                INSERT INTO results_{method_label}
                (n_shape, n_angle, id, sub_id, error)
                VALUES (?, ?, ?, ?, ?)
                """,
            [(n_shape, n_angle, _id, sub_id, err)
             for _, n_shape, n_angle, _id, sub_id, err in res]
        )

        cursor.execute(
            "INSERT INTO total_results (method_id, n_shape, n_angle, id) VALUES (?, ?, ?, ?)",
            (method_id, n_shape, n_angle, _id)
        )

        conn.commit()

conn.close()'''

#
# Single process version :
```

```
#

#
# Version sans multiprocessing
#

for param in params:
    method_id, n_shape, n_angle, _id, res = mesure_process(param)

    # Skip if already done
    cursor.execute(
        "SELECT 1 FROM total_results WHERE method_id=? AND n_shape=? AND n_angle=? AND id=?",
        (method_id, n_shape, n_angle, _id)
    )
    if cursor.fetchone():
        print(f"Already done: {method_id} {n_shape} {n_angle} {_id}")
        continue

    # Cleanup partial runs
    method_label = [k for k, v in METHODS_IDS.items() if v == method_id][0]
    cursor.execute(
        f"DELETE FROM results_{method_label} WHERE n_shape=? AND n_angle=? AND id=?",
        (n_shape, n_angle, _id)
    )

    # Bulk insert
    cursor.executemany(
        f"""
        INSERT INTO results_{method_label}
        (n_shape, n_angle, id, sub_id, error)
        VALUES (?, ?, ?, ?, ?)
        """,
        [
            (n_shape, n_angle, _id, sub_id, err)
            for _, n_shape, n_angle, _id, sub_id, err in res
        ]
    )

    cursor.execute(
        "INSERT INTO total_results (method_id, n_shape, n_angle, id) VALUES (?, ?, ?, ?)",
        (method_id, n_shape, n_angle, _id)
```

```
        )
        conn.commit()
conn.close()
```

results/4 illustration.py

```
# %%
import numpy as np
from scipy.ndimage import rotate
from plotly.subplots import make_subplots
import plotly.graph_objects as go

N = 128

img = np.zeros((N, N))

# barre horizontale
img[20:35, 30:98] = 1

# barre verticale
img[35:100, 54:74] = 1

img = np.rot90(img, 2)

angles = [0, 45, 90, 135]

backprojections = []

for theta in angles:

    # Projection
    rotated = rotate(
        img,
        theta,
        reshape=False,
        order=1
    )

    projection = rotated.sum(axis=0)
    bp = np.tile(projection, (N, 1))

    bp = rotate(
        bp,
        -theta,
        reshape=False,
        order=1
    )
```

```
    backprojections.append(bp)
reconstruction = np.sum(backprojections, axis=0)

reconstruction /= reconstruction.max()

fig = make_subplots(
    rows=2,
    cols=3,
    subplot_titles=[
        r"$\large{\text{Objet}}$",
        r"$\large{0°}$",
        r"$\large{45°}$",
        r"$\large{90°}$",
        r"$\large{135°}$",
        r"$\large{\text{Somme}}$"
    ]
)

# Objet original
fig.add_trace(
    go.Heatmap(
        z=img,
        colorscale="Greys",
        showscale=False
    ),
    row=1,
    col=1
)

# Back projections
positions = [
    (1, 2),
    (1, 3),
    (2, 1),
    (2, 2)
]

for bp, pos in zip(backprojections, positions):

    fig.add_trace(
```

```
        go.Heatmap(
            z=bp,
            colorscale="gray",
            showscale=False
        ),
        row=pos[0],
        col=pos[1]
    )

# Somme
fig.add_trace(
    go.Heatmap(
        z=reconstruction,
        colorscale="gray",
        showscale=False
    ),
    row=2,
    col=3
)

for i in range(1, 3):
    for j in range(1, 4):
        fig.update_xaxes(showticklabels=False, row=i, col=j)
        fig.update_yaxes(showticklabels=False, row=i, col=j)

fig.update_layout(
    template="plotly_white",

)

fig.write_image("images/5 fig1 lr.webp", scale=1.3, height=600, width=1038)
fig.write_image("images/5 fig1 hr.png", scale=10, height=600, width=1038)
# %%
import numpy as np
from scipy.ndimage import rotate
from plotly.subplots import make_subplots
import plotly.graph_objects as go

N = 128
img = np.zeros((N, N))
img[20:35, 30:98] = 1   # barre horizontale
```

```
img[35:100, 54:74] = 1   # barre verticale


angles = np.arange(0, 180, 5)   # 36 projections pour bien remplir l'espace
F_domain = np.zeros((N, N), dtype=complex)
center = N // 2

# Vecteur de coordonnées pour la tranche 1D
r = np.arange(-center, center)

for theta in angles:
    # 1. Projection
    rotated = rotate(img, theta, reshape=False, order=1)
    projection = rotated.sum(axis=0)

    # 2. Transformée de Fourier 1D de la projection
    # fftshift recentre les basses fréquences au milieu
    proj_fft = np.fft.fftshift(np.fft.fft(np.fft.ifftshift(projection)))

    # 3. Placement radial dans l'espace de Fourier 2D
    theta_rad = np.deg2rad(theta)

    # Conversion polaire -> cartésien (Arrondi au pixel le plus proche pour l'illustration)
    x = np.round(center + r * np.cos(theta_rad)).astype(int)
    y = np.round(center - r * np.sin(theta_rad)).astype(int)   # - car l'axe y descend

    # On ne garde que les indices qui restent dans l'image
    valid = (x >= 0) & (x < N) & (y >= 0) & (y < N)

    # On "dessine" la ligne de fréquences dans notre matrice 2D
    F_domain[y[valid], x[valid]] = proj_fft[valid]

reconstruction = np.fft.fftshift(np.fft.ifft2(np.fft.ifftshift(F_domain))).real
reconstruction = np.maximum(reconstruction, 0)   # Supprime les petits artefacts négatifs
reconstruction /= reconstruction.max()

F_original = np.fft.fftshift(np.fft.fft2(np.fft.ifftshift(img)))

fig = make_subplots(
    rows=2,
    cols=2,
```

```
    subplot_titles=[
        r"$\large{\text{Objet Original}}$",
        r"$\large{\text{Spectre Réel}}$",
        r"$\large{\text{Espace de Fourier en étoile}}$",
        r"$\large{\text{Reconstruction Spatiale}}$"
    ]
)

# 1. Objet Original
fig.add_trace(go.Heatmap(z=np.rot90(img, 2), colorscale="Greys", showscale=False), row=1, col=1)

# 2. Spectre Original (on utilise np.log pour voir les contrastes des fréquences)
fig.add_trace(go.Heatmap(z=np.log(1 + np.abs(F_original)), colorscale="Viridis", showscale=False), row=1,
col=2)

# 3. Espace de Fourier Reconstruit (Les fameuses "tranches")
fig.add_trace(go.Heatmap(z=np.log(1 + np.abs(F_domain)), colorscale="Viridis", showscale=False), row=2,
col=1)

# 4. Image Reconstruite
fig.add_trace(go.Heatmap(z=reconstruction, colorscale="RdYlGn", showscale=False), row=2, col=2)

# Nettoyage des axes
for i in range(1, 3):
    for j in range(1, 3):
        fig.update_xaxes(showticklabels=False, row=i, col=j)
        fig.update_yaxes(showticklabels=False, row=i, col=j)

fig.update_layout(
    template="plotly_white",
)

fig.write_image("images/5 fig2 lr.webp", scale=1.3, width=1170 / 2, height=1213 / 2)
fig.write_image("images/5 fig2 hr.png", width=1170 / 2, height=1213 / 2)
```

neural_networks/__init__.py

```python
__import__("os").environ["KERAS_BACKEND"] = "jax"
__import__("os").environ["XLA_PYTHON_CLIENT_PREALLOCATE"] = "false"


import os
import random

import numpy as np


def train(
        model,
        name: str,
        folder: str,
        batch_size: int = 16,
        epochs: int = 10,
        models_dir: str = "models",
):


    os.makedirs(models_dir, exist_ok=True)

    files = os.listdir(folder)
    view = [*map(lambda x: int(x[:-4].split("-")[3]), files)]
    obj = [*map(lambda x: int(x[:-4].split("-")[1]), files)]

    dict_data = {x: [] for x in set(view)}

    for f, v, o in zip(files, view, obj):
        dict_data[v].append(f)

    for v in dict_data.keys():
        random.shuffle(dict_data[v])


    iteration = 0

    while sum([len(v) for v in dict_data.values()]) > 0:
        r = random.choice([*filter(lambda v: len(dict_data[v]) > 0, dict_data.keys())])

        x_train, y_train = [], []
```

```
mem = 0.0
n_angles = 0

print("Iteration: ", iteration)
print(r)

while mem < 10000000000 and len(dict_data[r]) > 0:
    f_name = dict_data[r].pop()
    print(f"\r{f_name}\t {mem / 1_000_000}Mo", end="")

    data = np.load(os.path.join(folder, f_name))

    x_train.append(data["x"])
    y_train.append(data["y"])

    n_angles = data["x"].shape[1]

    mem += 2 * data["x"].nbytes + data["y"].nbytes

x_train = np.concatenate(x_train, axis=0)
y_train = np.concatenate(y_train, axis=0)
angles = np.linspace(0, 360, n_angles, endpoint=False, dtype=np.float32)

model.train(
    sinograms=x_train,
    angles_deg=angles,
    images=y_train,
    batch_size=batch_size,
    epochs=epochs,
)

del x_train, y_train

save_path = os.path.join(models_dir, f"{name}_{iteration:04d}.keras")
model.save(save_path)

iteration += 1
```

neural_networks/direct_reconstruction.py

```
__import__("os").environ["KERAS_BACKEND"] = "jax"


import numpy as np
import keras
from keras import layers, Model, ops


class Deg2Rad(layers.Layer):
    def call(self, angles_deg):
        return angles_deg * (np.pi / 180.0)

    def compute_output_shape(self, input_shape):
        return input_shape


class AngleEncoding(layers.Layer):
    def call(self, angles_rad):
        return ops.stack([ops.sin(angles_rad), ops.cos(angles_rad)], axis=-1)

     def compute_output_shape(self, input_shape):
         return input_shape[0], input_shape[1], 2


class CTReconstructionNetwork:
    def __init__(
            self,
            projection_dim: int = 128,   # taille embedding par projection
            latent_dim: int = 256,   # taille du vecteur global après pooling
            learning_rate: float = 1e-3,
    ):
        self.projection_dim = projection_dim
        self.latent_dim = latent_dim
        self.learning_rate = learning_rate
        self.model: Model = self._build()
        self._compile()

    def _build(self) -> Model:
        N = 100  # nb de détecteurs

        sinogram_in = keras.Input(shape=(None, N), name="sinogram")    # (B, n_angles, 100)
```

```
angles_in = keras.Input(shape=(None,), name="angles_deg")   # (B, n_angles)

# Encodage des angles
angles_rad = Deg2Rad(name="deg2rad")(angles_in)   # (B, n_angles)

angle_enc = AngleEncoding(name="angle_enc")(angles_rad)   # (B, n_angles, 2)

# Concaténation projection + angle
x = layers.Concatenate(axis=-1, name="proj_with_angle")(
    [sinogram_in, angle_enc]
)  # (B, n_angles, 102)

# MLP partagé sur chaque projection (TimeDistributed)
x = layers.TimeDistributed(layers.Dense(256, activation="relu"),
                           name="td_dense1")(x)   # (B, n_angles, 256)
 x = layers.TimeDistributed(layers.Dense(self.projection_dim, activation="relu"),
                           name="td_dense2")(x)   # (B, n_angles, projection_dim)

# Agrégation
z = layers.GlobalAveragePooling1D(name="angle_pooling")(x)   # (B, projection_dim)

# Couches denses de transition : espace latent
z = layers.Dense(self.latent_dim, activation="relu", name="latent_dense1")(z)
z = layers.Dense(self.latent_dim, activation="relu", name="latent_dense2")(z)

# Décodeur CNN transposé 5 -> 10 -> 20 -> 50 -> 100
z = layers.Dense(5 * 5 * 128, activation="relu", name="pre_reshape")(z)
z = layers.Reshape((5, 5, 128), name="reshape")(z)

z = layers.Conv2DTranspose(128, kernel_size=3, strides=2,
                           padding="same", activation="relu",
                           name="deconv1")(z)   # 10×10
z = layers.Conv2DTranspose(64, kernel_size=3, strides=2,
                           padding="same", activation="relu",
                           name="deconv2")(z)   # 20×20
z = layers.Conv2DTranspose(32, kernel_size=3, strides=2,
                           padding="same", activation="relu",
                           name="deconv3")(z)   # 40×40 (pad crop needed)
z = layers.Conv2DTranspose(16, kernel_size=3, strides=2,
                           padding="same", activation="relu",
                           name="deconv4")(z)   # 80×80
```

```
    # 80 -> 100 : un Conv2DTranspose avec stride 1 + crop/resize
    z = layers.UpSampling2D(size=(2, 2), name="upsample")(z)   # 160×160
    z = layers.Conv2D(16, kernel_size=3, padding="same",
                      activation="relu", name="refine1")(z)
    z = layers.Cropping2D(cropping=30, name="crop_to_100")(z)   # 100×100

    output = layers.Conv2D(1, kernel_size=1, activation="sigmoid",
                           name="output")(z)   # (B, 100, 100, 1)

    return Model(inputs=[sinogram_in, angles_in],
                 outputs=output,
                 name="CTReconNet")

def _compile(self):
    self.model.compile(
        optimizer=keras.optimizers.Adam(self.learning_rate),
        loss="mse",
        metrics=["mae"],
    )

def summary(self):
    self.model.summary()

def train(
        self,
        sinograms: np.ndarray,   # (N_samples, n_angles, 100)
        angles_deg: np.ndarray,   # (n_angles,) ou (N_samples, n_angles)
        images: np.ndarray,   # (N_samples, 100, 100)
        epochs: int = 50,
        batch_size: int = 16,
        validation_split: float = 0.1,
        vram_limit_gb: float = 10.0,
) -> None:
    N_samples = sinograms.shape[0]

    # Broadcast angles si nécessaire
    if angles_deg.ndim == 1:
        angles_input = np.tile(angles_deg[np.newaxis, :], (N_samples, 1)).astype(np.float32)
    else:
        angles_input = angles_deg.astype(np.float32)
```

```
# Normalisation sinogrammes
x_sino = sinograms.astype(np.float32)
smax = x_sino.max(axis=(1, 2), keepdims=True)
smax = np.where(smax == 0, 1.0, smax)
x_sino /= smax

# Normalisation images [0, 1]
y = images.astype(np.float32)
vmax = y.max(axis=(1, 2), keepdims=True)
vmax = np.where(vmax == 0, 1.0, vmax)
y = (y / vmax)[..., np.newaxis]   # (N, 100, 100, 1)

# Taille en octets d'un sample (sinogram + angles + image)
bytes_per_sample = (
        x_sino[0].nbytes + angles_input[0].nbytes + y[0].nbytes
)
chunk_size = max(batch_size, int(vram_limit_gb * 1e9 / bytes_per_sample))
chunk_size = min(chunk_size, N_samples)

# Shuffle global
idx = np.random.permutation(N_samples)
x_sino = x_sino[idx]
angles_input = angles_input[idx]
y = y[idx]

callbacks = [
    keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss", factor=0.5, patience=5, verbose=1
    ),
]

# Itération sur les chunks : model.fit() ne voit jamais plus de chunk_size samples
for start in range(0, N_samples, chunk_size):
    sl = slice(start, start + chunk_size)
    print(f"  chunk [{start}:{start + chunk_size}] / {N_samples}")
    self.model.fit(
        x={"sinogram": x_sino[sl], "angles_deg": angles_input[sl]},
        y=y[sl],
        epochs=epochs,
        batch_size=batch_size,
```

```
            validation_split=validation_split,
            callbacks=callbacks,
            verbose=1,
        )

def predict(
        self,
        sinogram: np.ndarray,   # (n_angles, 100)  ou  (B, n_angles, 100)
        angles_deg: np.ndarray,   # (n_angles,)       ou  (B, n_angles)
) -> np.ndarray:
    """
    Reconstruit une (ou plusieurs) image(s).

    Retourne : np.ndarray de shape (100, 100) si entrée 2D,
               (B, 100, 100) si entrée 3D.
    """
    single = sinogram.ndim == 2
    if single:
        sinogram = sinogram[np.newaxis]   # (1, n_angles, 100)
        angles_deg = angles_deg[np.newaxis]   # (1, n_angles)

    # Normalisation
    x_sino = sinogram.astype(np.float32)
    smax = x_sino.max(axis=(1, 2), keepdims=True)
    smax = np.where(smax == 0, 1.0, smax)
    x_sino = x_sino / smax

    pred = self.model.predict(
        {"sinogram": x_sino, "angles_deg": angles_deg.astype(np.float32)},
        verbose=0,
    )  # (B, 100, 100, 1)

    pred = pred[..., 0]  # (B, 100, 100)
    return pred[0] if single else pred

def save(self, path: str):
    self.model.save(path)

@classmethod
def load(cls, path: str) -> "CTReconstructionNetwork":
    obj = cls.__new__(cls)
```

```
        obj.model = keras.models.load_model(
            path,
            custom_objects={"AngleEncoding": AngleEncoding, "Deg2Rad": Deg2Rad},
        )
        return obj


if __name__ == "__main__":
    from __init__ import train

    model = CTReconstructionNetwork(projection_dim=128, latent_dim=256)

    train(
        model=model,
        name="Direct",
        folder=r"/mnt/c/Users/louis/Documents/Python/ADVANCED/Tomographie/data/training",
        batch_size=16,
        epochs=10,
        models_dir="models",
    )
```

neural_networks/unet.py

```python
from os import name

__import__("os").environ["KERAS_BACKEND"] = "jax"

import numpy as np
import keras
from keras import layers, Model, ops
import random
import os
from numba import njit, prange
import math


@njit(parallel=True, cache=True)
def _fast_backprojection(sinogram: np.ndarray, angles_deg: np.ndarray) -> np.ndarray:
    N = sinogram.shape[0]
    n_angles = sinogram.shape[1]
    det = sinogram.shape[2]
    cx = 50.0

    recon = np.zeros((N, 100, 100), dtype=np.float32)

    for n in prange(N):  # parallèle sur les samples
        for i in range(n_angles):
            rad = angles_deg[i] * (math.pi / 180.0)
            cos_a = math.cos(rad)
            sin_a = math.sin(rad)
            for row in range(100):
                for col in range(100):
                    x = col - cx
                    y = row - cx
                    t = x * cos_a + y * sin_a + cx
                    t_idx = int(t)
                    if t_idx < 0:
                        t_idx = 0
                    elif t_idx >= det:
                        t_idx = det - 1
                    recon[n, row, col] += sinogram[n, i, t_idx]

        for row in range(100):
            for col in range(100):
```

```
                recon[n, row, col] /= n_angles

    return recon


def _ramp_filter_fft(sinogram: np.ndarray) -> np.ndarray:
    det = sinogram.shape[2]
    freq = np.abs(np.fft.fftfreq(det)).astype(np.float32)   # (100,)

    # FFT sur l'axe détecteur (axis=2), multiplication, IFFT — tout vectorisé
    sino_fft = np.fft.fft(sinogram, axis=2)   # (N, n_angles, 100)
    sino_filtered = np.real(np.fft.ifft(sino_fft * freq, axis=2))
    return sino_filtered.astype(np.float32)


@njit(parallel=True, cache=True)
def _backproject_fft(sinogram: np.ndarray, angles_deg: np.ndarray) -> np.ndarray:
    N = sinogram.shape[0]
    n_angles = sinogram.shape[1]
    det = sinogram.shape[2]
    cx = 50.0

    recon = np.zeros((N, 100, 100), dtype=np.float32)

    for n in prange(N):
        for i in range(n_angles):
            rad = angles_deg[i] * (math.pi / 180.0)
            cos_a = math.cos(rad)
            sin_a = math.sin(rad)
            for row in range(100):
                for col in range(100):
                    t = (col - cx) * cos_a + (row - cx) * sin_a + cx
                    t_idx = int(t)
                    if t_idx < 0:
                        t_idx = 0
                    elif t_idx >= det:
                        t_idx = det - 1
                    recon[n, row, col] += sinogram[n, i, t_idx]

        for row in range(100):
            for col in range(100):
```

```
                recon[n, row, col] /= n_angles

    return recon


def _fast_backprojection_fft(sinogram: np.ndarray, angles_deg: np.ndarray) -> np.ndarray:
    return _backproject_fft(_ramp_filter_fft(sinogram), angles_deg)


_fast_backprojection(np.zeros((3, 100, 100)), np.linspace(0, 360, 100))
_fast_backprojection_fft(np.zeros((3, 100, 100)), np.linspace(0, 360, 100))


class CTUNet:
    def __init__(
            self,
            filters: int = 32,   # filtres de base (doublés à chaque niveau)
            depth: int = 4,   # nombre de niveaux encodeur/décodeur
            learning_rate: float = 1e-3,
            bp_type: str ="fft"
    ):
        self.filters = filters
        self.depth = depth
        self.learning_rate = learning_rate
        self.model: Model = self._build()
        self._compile()
        self.bp_type = bp_type

    @staticmethod
    def _conv_block(x, n_filters: int, name: str):
        x = layers.Conv2D(n_filters, 3, padding="same", name=f"{name}_c1")(x)
        x = layers.BatchNormalization(name=f"{name}_bn1")(x)
        x = layers.Activation("relu", name=f"{name}_r1")(x)
        x = layers.Conv2D(n_filters, 3, padding="same", name=f"{name}_c2")(x)
        x = layers.BatchNormalization(name=f"{name}_bn2")(x)
        x = layers.Activation("relu", name=f"{name}_r2")(x)
        return x

    @staticmethod
    def _match_and_concat(x, skip, name: str):
        # shapes symboliques
```

```
    xs = ops.shape(x)
    ss = ops.shape(skip)
    # crop sur h et w si x est plus grand que skip
    dh = xs[1] - ss[1]
    dw = xs[2] - ss[2]
    if dh > 0 or dw > 0:
        x = layers.Cropping2D(
            cropping=((0, dh), (0, dw)), name=f"{name}_crop"
        )(x)
    return layers.Concatenate(name=name)([x, skip])

def _build(self) -> Model:
    inp = keras.Input(shape=(100, 100, 1), name="bp_image")

    # Encodeur
    skips = []
    x = inp
    for d in range(self.depth):
        f = self.filters * (2 ** d)
        x = self._conv_block(x, f, name=f"enc{d}")
        skips.append(x)
        x = layers.MaxPooling2D(2, padding="same", name=f"pool{d}")(x)

    # Bottleneck
    x = self._conv_block(x, self.filters * (2 ** self.depth), name="bottleneck")

    # Décodeur
    for d in reversed(range(self.depth)):
        f = self.filters * (2 ** d)
        x = layers.Conv2DTranspose(f, 2, strides=2, padding="same",
                                   name=f"up{d}")(x)
        x = self._match_and_concat(x, skips[d], name=f"skip{d}")
        x = self._conv_block(x, f, name=f"dec{d}")

    # Sortie résiduelle : UNet prédit la correction
    correction = layers.Conv2D(1, 1, activation="tanh", name="correction")(x)
    output = layers.Add(name="output")([inp, correction])
    output = layers.Activation("sigmoid", name="clip")(output)

    return Model(inputs=inp, outputs=output, name="CTUNet")
```

```
def _compile(self):
    self.model.compile(
        optimizer=keras.optimizers.Adam(self.learning_rate),
        loss="mse",
        metrics=["mae"],
    )

def summary(self):
    self.model.summary()

def _prepare_inputs(
        self,
        sinograms: np.ndarray,   # (N, n_angles, 100)
        angles_deg: np.ndarray,   # (n_angles,)
) -> np.ndarray:
    if self.bp_type  == "bp":
        bp = _fast_backprojection(sinograms, angles_deg)   # (N, 100, 100)

    elif self.bp_type  == "fft":
        bp = _fast_backprojection_fft(sinograms, angles_deg)

    else:
        raise Exception()

    # Normalisation par sample
    vmax = bp.max(axis=(1, 2), keepdims=True)
    vmax = np.where(vmax == 0, 1.0, vmax)
    bp /= vmax
    return bp[..., np.newaxis]   # (N, 100, 100, 1)

def train(
        self,
        sinograms: np.ndarray,   # (N, n_angles, 100)
        angles_deg: np.ndarray,   # (n_angles,)
        images: np.ndarray,   # (N, 100, 100)
        epochs: int = 50,
        batch_size: int = 16,
        validation_split: float = 0.1,
        vram_limit_gb: float = 10.0,
) -> None:
    print("Backprojection...", end=" ", flush=True)
```

```
    x = self._prepare_inputs(sinograms, angles_deg,)
    print("OK")

    # Normalisation images cibles
    y = images.astype(np.float32)
    vmax = y.max(axis=(1, 2), keepdims=True)
    vmax = np.where(vmax == 0, 1.0, vmax)
    y = (y / vmax)[..., np.newaxis]   # (N, 100, 100, 1)

    # Chunk pour ne pas dépasser vram_limit_gb
    bytes_per_sample = x[0].nbytes + y[0].nbytes
    chunk_size = max(batch_size, int(vram_limit_gb * 1e9 / bytes_per_sample))
    chunk_size = min(chunk_size, len(x))

    idx = np.random.permutation(len(x))
    x, y = x[idx], y[idx]

    callbacks = [
        keras.callbacks.ReduceLROnPlateau(
            monitor="val_loss", factor=0.5, patience=5, verbose=1
        ),
    ]

    for start in range(0, len(x), chunk_size):
        sl = slice(start, start + chunk_size)
        print(f"  chunk [{start}:{min(start + chunk_size, len(x))}] / {len(x)}")
        self.model.fit(
            x=x[sl], y=y[sl],
            epochs=epochs,
            batch_size=batch_size,
            validation_split=validation_split,
            callbacks=callbacks,
            verbose=1,
        )

def predict(
        self,
        sinogram: np.ndarray,   # (n_angles, 100)  ou  (N, n_angles, 100)
        angles_deg: np.ndarray,   # (n_angles,)
) -> np.ndarray:
    single = sinogram.ndim == 2
```

```
        if single:
            sinogram = sinogram[np.newaxis]

        x = self._prepare_inputs(sinogram, angles_deg)   # (N, 100, 100, 1)
        pred = self.model.predict(x, verbose=0)   # (N, 100, 100, 1)
        pred = pred[..., 0]   # (N, 100, 100)
        return pred[0] if single else pred

    def save(self, path: str):
        self.model.save(path)

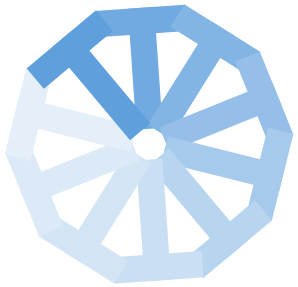
    @classmethod
    def load(cls, path: str) -> "CTUNet":
        obj = cls.__new__(cls)
        obj.model = keras.models.load_model(path)
        return obj
```

```
if __name__ == "__main__":
    BP_TYPE = "fft"

    from __init__ import train

    model = CTUNet(bp_type=BP_TYPE)

    train(
        model=model,
        name="UNet" + ("" if BP_TYPE == "bp" else "_fft"),
        folder=r"/mnt/c/Users/louis/Documents/Python/ADVANCED/Tomographie/data/training",
        batch_size=16,
        epochs=10,
        models_dir="models",
    )
```



RECONSTRUCTION TOMOGRAPHIQUE

TIPE | Louis H. | 00000